

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

К защите допустить:

Заведующий кафедрой ИИТ

_____ Д. В. Шункевич

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
по дисциплине «Модели решения задач в интеллектуальных системах»
на тему:

**ПОЛЬЗОВАТЕЛЬСКИЙ ИНТЕРФЕЙС
МЕТАСИСТЕМЫ OSTIS**

БГУИР КП6 6-05-0611-03 046 ПЗ

Студент гр. 321701
Проверяющий

В. А. Лукашов
Н. В. Зотов

Минск 2026

СОДЕРЖАНИЕ

Перечень условных обозначений	6
Введение	7
1 Анализ подходов к разработке интерфейса интеллектуальной системы	9
1.1 Характеристика предметной области	9
1.2 Участники взаимодействия и их задачи	10
1.3 Анализ существующих систем и интерфейсов	11
1.4 Анализ требований к интерфейсу	15
1.5 Вывод по разделу анализа	16
2 Проектирование интеллектуальной системы	18
2.1 Общая архитектура интеллектуальной системы	18
2.2 Архитектура интерфейсного слоя	18
2.3 Модели пользователей и сценарии использования	22
2.4 Проектирование пользовательского интерфейса	31
2.5 Проектирование программного интерфейса	33
2.6 Проектирование интеллектуальных механизмов интерфейса	35
2.7 Вывод по разделу проектирования	36
3 Реализация интеллектуальной системы	37
3.1 Выбор средств реализации	37
3.2 Реализация программного интерфейса	39
3.3 Тестирование интерфейса	42
3.4 Вывод по разделу реализации	44
Заключение	46
Список использованных источников	48

ПЕРЕЧЕНЬ УСЛОВНЫХ ОБОЗНАЧЕНИЙ

В курсовой работе используются следующие условные обозначения:

- БЗ — База знаний;
- AI (от англ. Artificial Intelligence) — Искусственный интеллект;
- API (от англ. Application Programming Interface) — Прикладной программный интерфейс;
- CSS (от англ. Cascading Style Sheets) — Каскадные таблицы стилей;
- DOM (от англ. Document Object Model) — Объектная модель документа;
- HTML (от англ. HyperText Markup Language) — Язык разметки гипертекста;
- HTTP (от англ. HyperText Transfer Protocol) — Протокол передачи гипертекста;
- JSON (от англ. JavaScript Object Notation) — Текстовый формат обмена данными, основанный на JavaScript;
- JSX (от англ. JavaScript XML) — Расширение синтаксиса JavaScript;
- OSTIS (от англ. Open Semantic Technology for Intelligent Systems) — Открытая Семантическая Технология Интеллектуальных Систем;
- REST (от англ. Representational State Transfer) — Передача состояния представления;
- SC (от англ. Semantic Code) — Семантический код;
- SCg (от англ. Semantic Code graphical) — Графический семантический код;
- SCn (от англ. Semantic Code natural) — Естественный семантический код;
- SCs (от англ. Semantic Code string) — Семантическая кодовая строка;
- UI (от англ. User Interface) — Пользовательский интерфейс;
- URL (от англ. Uniform Resource Locator) — Уникальный унифицированный адрес;
- UTF-8 (от англ. Unicode Transformation Format, 8-bit) — Формат преобразования Юникода, 8-битная кодировка.

ВВЕДЕНИЕ

Метасистема *OSTIS* представляет собой инструментальную среду для разработки и сопровождения интеллектуальных систем, основанных на принципах семантического представления знаний. В рамках данной технологии ключевую роль играет использование формализованных моделей знаний, позволяющих описывать предметные области, процессы и взаимосвязи между объектами. Однако практическое применение Метасистемы *OSTIS* сопряжено с рядом сложностей, обусловленных высокой степенью формализации и необходимостью работы с низкоуровневыми структурами представления знаний.

Актуальность разработки пользовательского интерфейса интеллектуальной системы обусловлена возрастающей сложностью современных систем обработки и представления знаний. Метасистема *OSTIS* ориентирована на создание и сопровождение интеллектуальных систем на основе формализованных семантических моделей, что предполагает активное взаимодействие пользователя с базой знаний и механизмами вывода. При этом низкоуровневый характер представления знаний и высокая насыщенность функциональными возможностями существенно усложняют процесс работы с системой, особенно для новых пользователей. В этих условиях пользовательский интерфейс становится ключевым компонентом, определяющим доступность технологий интеллектуальных систем и эффективность их практического применения.

Разрабатываемый пользовательский интерфейс относится к предметной области интеллектуальных информационных систем и инженерии знаний. В рамках данной области решаются задачи накопления, структурирования, интерпретации и использования знаний, представленных в формализованном виде. Метасистема *OSTIS* используется как инструментальная среда для проектирования и эволюции интеллектуальных систем, что предполагает выполнение таких задач, как навигация по семантическим структурам, визуализация фрагментов базы знаний, формирование запросов и анализ полученных результатов. Пользовательский интерфейс в этом контексте выступает связующим звеном между внутренними механизмами системы и человеком, обеспечивая удобное и осмысленное взаимодействие с интеллектуальными компонентами.

Тема курсового проекта логически продолжает материалы, изученные в предыдущих семестрах, включая дисциплины, связанные с моделями представления знаний, архитектурой интеллектуальных систем и агентными механизмами обработки информации. Ранее основное внимание уделялось внутренним моделям и алгоритмам функционирования интеллектуальных систем, тогда как вопросы взаимодействия пользователя с системой рассматривались ограниченно. В рамках данного курсового проекта акцент

смещается на прикладной аспект — разработку пользовательского интерфейса, который опирается на ранее изученные теоретические основы и демонстрирует их практическое применение в рамках Метасистемы *OSTIS*.

Цель курсового проекта — разработка и обоснование пользовательского интерфейса Метасистемы *OSTIS*, обеспечивающего удобное и эффективное взаимодействие пользователя с интеллектуальными компонентами системы. Разрабатываемый интерфейс должен снижать сложность работы с формализованными семантическими структурами, поддерживать основные сценарии использования метасистемы и способствовать более наглядному представлению и управлению знаниями.

Задачи исследования:

- реализация возможности задания вопросов системе и компонентам системы;
- разработка диалогового компонента с возможностью задания естественно-языковых вопросов пользователя и получения естественно-языковых ответов на них;
- разработка компонентов для визуализации и редактирования разделов базы знаний;
- разработка компонентов навигации и редактирования компонентов Библиотеки *OSTIS*.

Объектом исследования и разработки в рамках курсового проекта является интеллектуальная система, реализуемая на базе Метасистемы *OSTIS*, предназначенной для хранения, обработки и использования формализованных знаний.

Предметом курсового проекта является пользовательский интерфейс данной интеллектуальной системы, включая архитектуру интерфейсного слоя, сценарии взаимодействия пользователя с системой, а также средства визуализации и управления элементами базы знаний.

Границы разработки определяются учебным характером курсового проекта и включают проектирование и частичную реализацию прототипа пользовательского интерфейса. В рамках работы рассматриваются основные функции навигации по компонентам системы, визуализации семантических структур и формирования пользовательских запросов. Реализация внутренних интеллектуальных механизмов, алгоритмов логического вывода и полноценной серверной части Метасистемы *OSTIS* не входит в рамки данного проекта и предполагается как уже существующая или абстрагированная функциональность.

Курсовой проект выполнен самостоятельно, проверен в системе «Антиплагиат». Процент оригинальности соответствует норме, установленной кафедрой ИИТ. Цитирования обозначены ссылками на публикации, указанные в «Списке использованных источников».

1 АНАЛИЗ ПОДХОДОВ К РАЗРАБОТКЕ ИНТЕРФЕЙСА ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ

1.1 Характеристика предметной области

Разрабатываемый пользовательский интерфейс функционирует в предметной области интеллектуальных систем и технологий представления знаний. В качестве базовой платформы используется Метасистема *OSTIS*[1], предназначенная для построения, сопровождения и развития интеллектуальных систем на основе формализованных семантических моделей. Данная предметная область характеризуется высокой степенью абстракции, использованием онтологических структур и необходимостью точного соответствия внутреннего представления знаний установленным стандартам.

Особенностью рассматриваемой области является применение низкоуровневых форм представления знаний, ориентированных прежде всего на машинную обработку. В связи с этим одной из ключевых задач становится преобразование сложных семантических конструкций в формы, доступные для восприятия и использования человеком. Пользовательский интерфейс в данном контексте выполняет роль средства абстрагирования, обеспечивая переход от формального описания знаний к наглядному и управляемому представлению.

В рамках Метасистемы *OSTIS* решаются задачи, связанные с навигацией по структурам базы знаний, анализом семантических связей между объектами, а также формированием и выполнением пользовательских запросов. Интерфейс должен поддерживать поиск и просмотр компонентов системы, отображение фрагментов базы знаний в различных формах и взаимодействие с интеллектуальными механизмами обработки информации. Ожидаемым результатом функционирования системы является повышение эффективности работы пользователей и упрощение процесса взаимодействия с интеллектуальными компонентами.

При проектировании интерфейса необходимо учитывать ряд ограничений и особенностей предметной области. К ним относятся требования к производительности, выражающиеся в допустимом времени отклика на действия пользователя и обработке запросов, а также необходимость строгого соблюдения стандартов представления и обмена данными, принятых в технологии *OSTIS*. Дополнительно учитываются требования к доступности и универсальности интерфейса, включая его корректную работу в различных программных и аппаратных средах, а также адаптацию к различным форматам отображения.

1.2 Участники взаимодействия и их задачи

Взаимодействие пользователей с интеллектуальной системой, реализованной на базе Метасистемы *OSTIS*, осуществляется в рамках распределённой архитектуры с разделением клиентской и серверной частей. В процессе функционирования системы задействованы несколько категорий участников, каждая из которых выполняет определённые функции и использует интерфейс в собственном контексте.

Разработчик интеллектуальных систем относится к категории конечных пользователей и использует систему в процессе создания и сопровождения компонентов базы знаний (*БЗ*). Основной целью данного участника является формирование, анализ и модификация семантических структур. В рамках своей деятельности разработчик осуществляет навигацию по библиотеке компонентов, просматривает и редактирует фрагменты *БЗ* в различных формах представления, а также формирует запросы к системе, включая запросы на естественном языке. В качестве входных данных используются семантические конструкции и пользовательские команды, а результатом взаимодействия являются визуализированные структуры и ответы интеллектуальных механизмов. Контекст использования интерфейса — рабочее место разработчика с доступом к системе через веб-браузер.

Администратор метасистемы также является конечным пользователем, однако его взаимодействие с системой носит управленческий характер. Целью администратора является обеспечение корректного функционирования системы, контроль изменений и управление правами доступа. В процессе работы администратор задаёт параметры доступа, рассматривает предложения на изменение компонентов базы знаний и анализирует информацию о состоянии системы. Интерфейс для данной роли используется в режиме административной панели и предполагает работу в защищённой среде с повышенными требованиями к надёжности и безопасности.

Платформа *OSTIS* выступает в роли внутренней подсистемы, обеспечивающей хранение знаний и выполнение интеллектуальной обработки. Она отвечает за управление внутренним представлением знаний, интерпретацию моделей пользовательского интерфейса и выполнение запросов, поступающих от клиентской части. В качестве входных данных платформа получает команды и запросы от интерфейса, а в качестве выходных — структурированные результаты обработки, предназначенные для последующей визуализации. Работа данной подсистемы осуществляется в серверной среде и не предполагает прямого взаимодействия с пользователем.

Клиентское устройство пользователя представляет собой техническое средство доступа к системе. Его основной задачей является отображение элементов пользовательского интерфейса, обработка пользовательских действий и передача соответствующих запросов на сервер. В рамках кли-

ентского окружения выполняется рендеринг интерфейсных компонентов, обработка событий ввода и визуализация полученных данных. Контекст использования включает современные веб-браузеры на настольных и мобильных устройствах.

Таким образом, основные сценарии взаимодействия реализуются через веб-интерфейс, в котором клиентская часть обеспечивает ввод и визуальное представление информации, а серверная — интеллектуальную обработку и управление знаниями. Конечные пользователи взаимодействуют с системой опосредованно, посредством интерфейсных компонентов, транслирующих их действия во внутренние операции Метасистемы *OSTIS*.

1.3 Анализ существующих систем и интерфейсов

Обзор аналогичных систем

Для выбора оптимальных архитектурных и интерфейсных решений Метасистемы *OSTIS* был выполнен анализ существующих программных продуктов, применяемых для визуализации данных, автоматизации рабочих процессов и управления персональными БЗ. Основной целью исследования стало сравнение инструментов, предлагающих различные методы организации, управления и представления семантических и структурированных данных. В качестве наиболее подходящих аналогов были выделены системы *Flowise*, *n8n* и *Logseq*, которые обеспечивают эффективное построение потоков данных, моделирование процессов и управление связями между объектами знаний. *Flowise* предоставляет веб-интерфейс, позволяющий пользователю визуально формировать последовательности действий посредством перетаскивания блоков и соединения их логическими связями, обеспечивая наглядное отображение процессов и ориентируясь на взаимодействие человек–система. *n8n* является open-source платформой для автоматизации процессов, предоставляющей возможность создавать и управлять цепочками операций между различными сервисами через графический веб-интерфейс, что облегчает интеграцию и визуализацию потоков данных. *Logseq* представляет собой персональную систему управления знаниями с блоковой структурой и двунаправленными связями между заметками, обеспечивая гибкую организацию информации и визуализацию графа знаний через веб- и desktop-приложения.

Анализ интерфейсов существующих решений

Детальное изучение пользовательских интерфейсов выбранных систем позволяет выявить характерные подходы к организации взаимодействия с данными, а также преимущества и ограничения, которые важно учитывать при проектировании интерфейса Метасистемы.

Интерфейс системы Flowise

Flowise [2] предоставляет веб-интерфейс (рисунок 1.1), ориентирован-

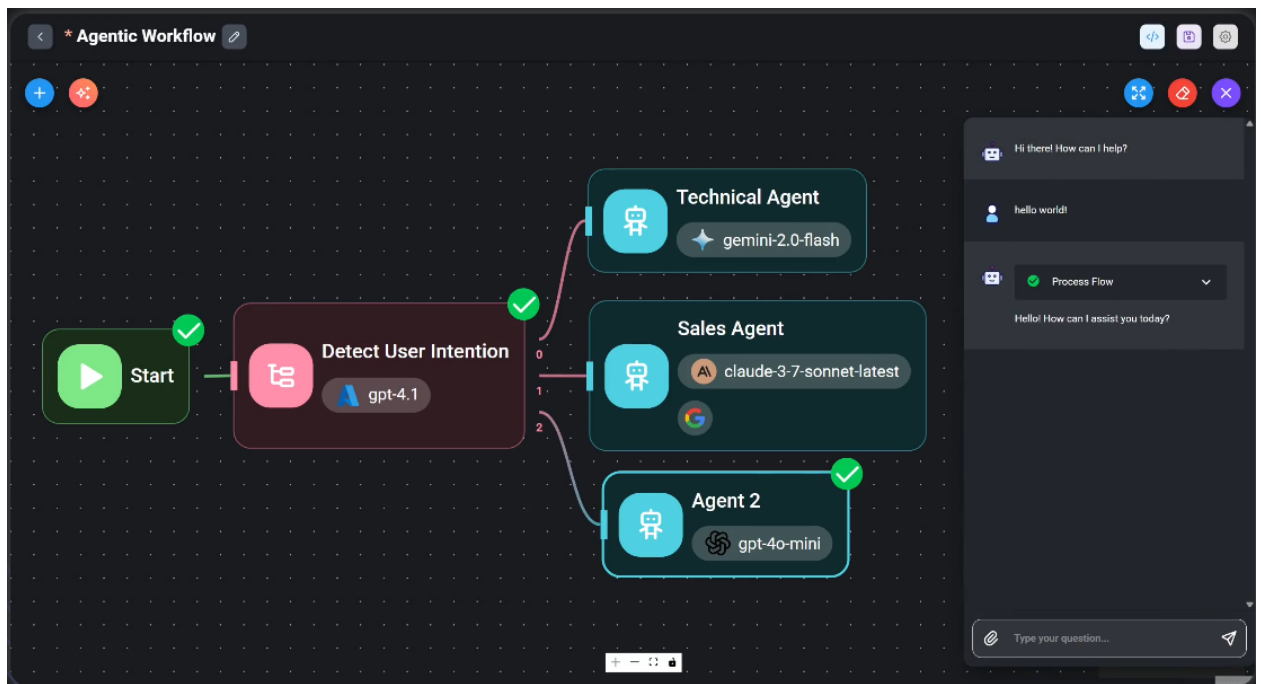


Рисунок 1.1 – Интерфейс платформы Flowise

ный на визуальное построение потоков данных и интеграцию различных сервисов. Основной рабочий экран системы организован по принципу «drag-and-drop», где пользователь располагает блоки операций на рабочей области и соединяет их логическими связями. Структура интерфейса включает панель доступных блоков, рабочую область для построения схем, а также панель свойств выбранного блока для настройки параметров. Ключевым сценарием взаимодействия является построение цепочек обработки данных, от источника до конечного результата, с возможностью тестирования и отладки на каждом этапе. Основными достоинствами интерфейса являются высокая наглядность процессов, простота перетаскивания элементов и возможность быстрого прототипирования. Однако среди недостатков можно отметить ограниченные возможности визуализации больших потоков данных и потенциальную перегруженность интерфейса при создании сложных схем.

Интерфейс системы n8n

Система n8n [3] представляет собой open-source платформу для автоматизации рабочих процессов, предоставляющую пользователю графический веб-интерфейс (рисунок 1.2). Основной рабочий экран содержит панель узлов, область построения цепочек действий и контекстную панель настроек каждого узла. Пользователь взаимодействует с системой через соединение блоков, настройку триггеров и параметров операций, а также тестирование выполнения процессов. Среди ключевых сценариев — интеграция внешних сервисов, автоматизация задач и управление потоками данных. Сильными сторонами интерфейса являются интуитивно понятное

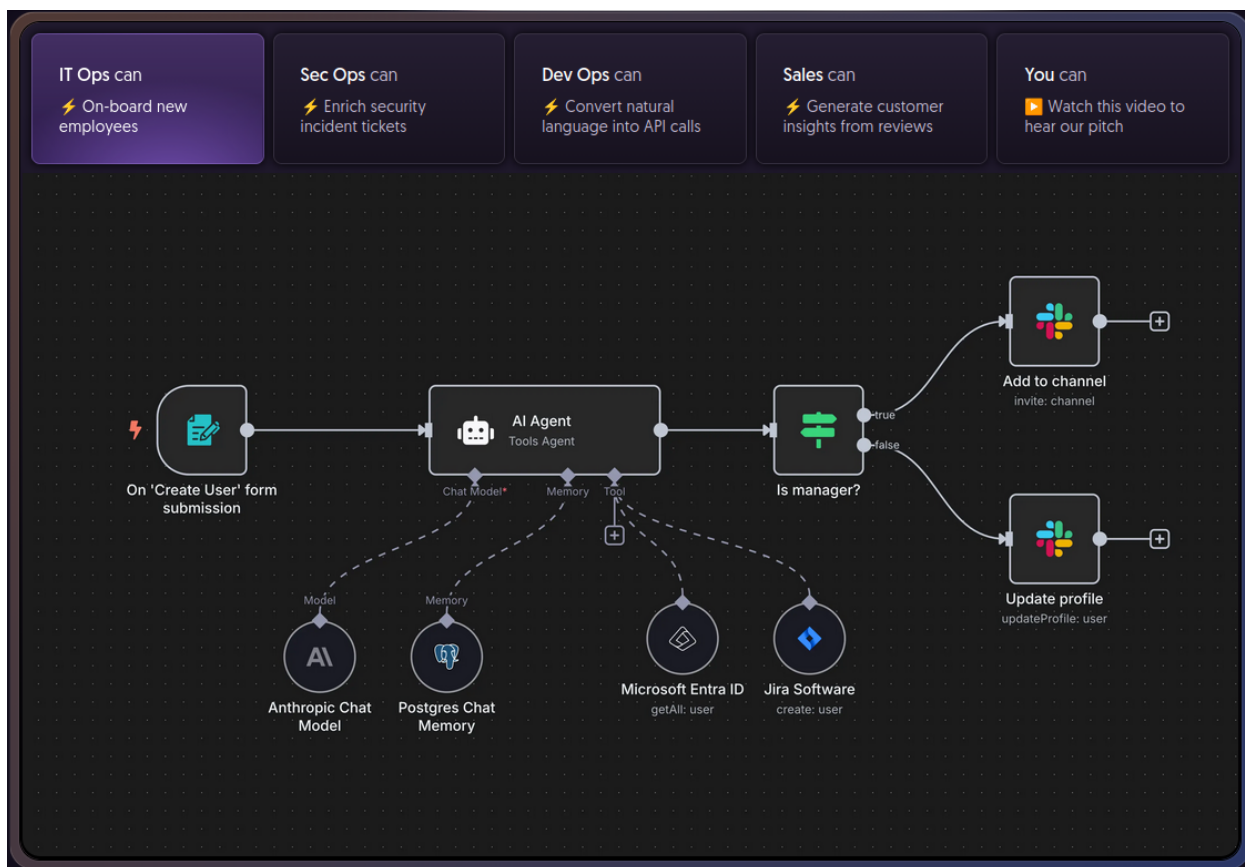


Рисунок 1.2 – Интерфейс системы n8n

построение процессов, прозрачная настройка узлов и наличие встроенных инструментов для отладки. Среди ограничений можно отметить необходимость понимания логики потоков данных для корректного соединения узлов, а также сравнительно высокую сложность работы с масштабными схемами без использования дополнительных визуальных фильтров.

Интерфейс системы Logseq

Logseq [4] является персональной БЗ (рисунок 1.3) с поддержкой блочной структуры и двунаправленных ссылок между заметками. Основной рабочий экран включает панель навигации, область редактирования блоков и граф знаний, отображающий связи между заметками. Типовые операции пользователя включают создание и связывание заметок, группировку информации по проектам, поиск и фильтрацию данных, а также визуализацию графа для анализа семантических связей. Ключевой особенностью интерфейса является гибкость организации информации и возможность персонализации структуры базы знаний под конкретные задачи. Сильными сторонами являются наглядная визуализация связей, возможность интеграции с Markdown и открытость платформы. К недостаткам можно отнести ограниченную автоматизацию процессов и необходимость привыкания к блочной структуре для пользователей, ранее не работавших с подобными системами.



Рисунок 1.3 – Интерфейс персональной базы знаний Logseq

Сравнительный анализ

Проведённый анализ показывает, что рассмотренные решения покрывают отдельные аспекты требований, предъявляемых к Метасистеме *OSTIS*, но ни одно из них не обеспечивает комплексного сочетания визуальной наглядности, гибкости управления знаниями и поддержки семантических связей. *Flowise* и *n8n* демонстрируют высокую наглядность процессов и потоков данных, позволяя пользователю быстро создавать и тестировать последовательности действий, однако их интерфейсы ориентированы преимущественно на моделирование процессов и автоматизацию, а не на управление семантическими связями и построение БЗ. В то же время *Logseq* обеспечивает гибкую организацию информации, поддержку двунаправленных ссылок и визуализацию графа знаний, но ограничен в возможностях автоматизации потоков и интеграции с внешними сервисами.

Интерфейс Метасистемы *OSTIS* должен объединять сильные стороны этих подходов: обеспечивать визуальную наглядность потоков и структур данных, предоставлять гибкие инструменты для управления связями между объектами знаний и поддерживать интерактивное построение процессов. Уникальными возможностями разрабатываемой системы станут интеграция инструментов автоматизации и визуального моделирования с семантической строгостью работы с типизированными связями SC-кода[1], поддержка коллективного редактирования с ролевой моделью и встроенные средства анализа и генерации дочерних систем, что позволит устранить ограничения

существующих аналогов и повысить эргономику веб-интерфейса.

1.4 Анализ требований к интерфейсу

Функциональные требования

Исходя из анализа предметной области и изучения интерфейсов существующих систем сформулированы требования к пользовательскому интерфейсу Метасистемы *OSTIS*. Интерфейс должен обеспечивать выполнение операций для различных ролей пользователей, таких как Разработчик, Администратор и Пользователь:

- **Авторизация и управление доступом:** Интерфейс должен обеспечивать форму входа в систему, регистрацию новых пользователей, управление правами доступа и отображение текущего статуса аутентификации. Для Администратора необходимо предусмотреть возможность назначения ролей и контроля уровня доступа к компонентам базы знаний.

- **Автоматизация процессов и интеграция:** Интерфейс должен обеспечивать инструменты для визуального моделирования процессов и потоков данных (по примеру *Flowise* и *n8n*), включая создание цепочек операций, настройку параметров и тестирование выполнения. Пользователь должен иметь возможность интегрировать внешние сервисы и просматривать результаты автоматизированных сценариев в наглядной форме.

- **Навигация:** Интерфейс должен предоставлять пользователю удобные средства навигации по структуре базы знаний, поддерживать поиск элементов по различным критериям (название, тип, семантические связи) и обеспечивать переход по связанным объектам без потери контекста.

- **Управление семантическими связями и графом знаний:** Интерфейс должен предоставлять средства для создания и редактирования связей между объектами знаний, визуализации графа связей (по примеру *Logseq*) и фильтрации информации для анализа. Это необходимо для поддержки гибкой структуры базы знаний и эффективного взаимодействия с семантическими данными.

Нефункциональные требования

Интерфейс также должен удовлетворять и следующие нефункциональные требования:

- **Скорость работы и отзывчивость.** Интерфейс должен обеспечивать быстрый отклик на действия пользователя (клик, перемещение, переключение режимов), выполнение сложных процессов и отображение графов знаний. Не допускается зависание или блокировка интерфейса при обработке больших потоков данных.

- **Доступность и кроссплатформенность:** Интерфейс следует обеспечивать в современных веб-браузерах (Chrome, Firefox, Microsoft Edge). Не допускается использование технологий, требующих установки специфич-

ческих плагинов или расширений.

- **Безопасность и конфиденциальность:** Следует обеспечить защиту данных пользователей и компонентов базы знаний, включая разграничение прав доступа и ведение журналов изменений. Не допускается хранение паролей и конфиденциальной информации в открытом виде.

Требования к «интеллектуальности» интерфейса

Интерфейс Метасистемы *OSTIS* должен включать элементы интеллектуального поведения, обеспечивающие помощь пользователю в работе с базой знаний и визуальными структурами:

- **Контекстная поддержка действий:** Интерфейс должен предоставлять рекомендации и подсказки на основе текущего состояния элементов базы знаний и действий пользователя. Например, при редактировании узлов или связей система должна предлагать допустимые варианты операций с учётом структуры графа.

- **Объяснение результатов операций:** При выполнении сложных действий (например, автоматической генерации связей или проверки согласованности данных) интерфейс должен отображать краткое пояснение, поясняющее пользователю, почему был выбран тот или иной результат, на основе семантических правил.

- **Адаптация интерфейса под пользователя:** Система должна сохранять индивидуальные настройки отображения и предпочтения пользователя, например, вид графа или степень детализации информации. Интерфейс должен корректно подстраиваться под задачи конкретной роли (Разработчик, Администратор, Пользователь).

- **Интеллектуальная обработка сообщений:** Интерфейс должен предоставлять уведомления и рекомендации в случаях ошибок или некорректных действий пользователя, формируя их на основе анализа текущего контекста и правил работы с базой знаний.

1.5 Вывод по разделу анализа

Проведённое исследование предметной области показало, что одной из основных трудностей при работе с Метасистемой *OSTIS* является сложность взаимодействия с семантическими структурами и потоками данных. Для пользователей важно создать интерфейс, который скрывает детали низкоуровневого SC-кода [1] и обеспечивает удобный доступ к информации в базе знаний. Специализированный пользовательский интерфейс становится ключевым инструментом для упрощения этих процессов и повышения эффективности работы с системой.

В процессе анализа участников взаимодействия были выделены основные роли: Разработчик, Администратор и Пользователь. Для каждой роли определены критически важные сценарии работы, включая авториза-

цию и управление учетными записями, поиск и навигацию по базе данных, редактирование и визуализацию элементов знаний, а также поддержку совместной работы и управления потоками данных.

Обзор существующих аналогов (*Flowise*, *n8n* и *Logseq*) показал, что ни одно решение не объединяет все необходимые возможности одновременно. *Flowise* и *n8n* предоставляют удобные средства для визуального построения и тестирования процессов, но их возможности по управлению связями и структурой знаний ограничены. *Logseq* позволяет эффективно работать с графом знаний и двунаправленными связями, однако не обеспечивает инструментов для автоматизации процессов и интеграции внешних сервисов.

На основании полученных данных при проектировании интерфейса Метасистемы *OSTIS* будут применены современные веб-технологии с модульной архитектурой, реализованы инструменты для визуального моделирования процессов и автоматизации операций, предусмотрена поддержка адаптивных подсказок и механизмов коллективного редактирования с ограничением прав пользователей. Эти решения позволят устранить недостатки существующих аналогов, повысить наглядность и удобство работы с базой знаний и обеспечить более эффективное взаимодействие с системой для всех категорий пользователей.

2 ПРОЕКТИРОВАНИЕ ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ

2.1 Общая архитектура интеллектуальной системы

Интеллектуальная система *OSTIS* имеет многокомпонентную структуру, включающую базу знаний, решатель задач, пользовательский интерфейс, а также средства взаимодействия с внешними сервисами. Такая организация позволяет разделить функции хранения, обработки и представления информации, обеспечивая гибкость и расширяемость системы.

Центральным элементом архитектуры является база знаний, которая представляется в виде семантической сети, реализованной средствами *SC-памяти*. В ней хранится информация о предметной области, структурах данных и взаимосвязях между объектами. Доступ к данным осуществляется через специализированные механизмы работы с *SC-графами*.

Обработка пользовательских запросов выполняется подсистемой решения задач, включающей набор программных компонентов, реализующих алгоритмы работы с данными. Данные компоненты анализируют входные запросы, выполняют необходимые преобразования и формируют результаты на основе содержимого базы знаний.

Пользовательский интерфейс реализован в виде веб-приложения и предназначен для обеспечения удобного взаимодействия пользователя с системой. Он отвечает за отображение данных, навигацию по структуре знаний, а также передачу пользовательских действий в подсистему обработки. Интерфейс не содержит логики обработки знаний, а выступает средством визуализации и управления.

Взаимодействие между интерфейсом и вычислительными компонентами осуществляется через промежуточный программный слой, обеспечивающий передачу запросов и получение результатов. Кроме того, архитектура предусматривает возможность подключения внешних сервисов, используемых для расширения функциональности системы, например, при экспорте данных или выполнении дополнительных операций.

Таким образом, интерфейс располагается на верхнем уровне архитектуры и обеспечивает доступ пользователя к функциональности системы, взаимодействуя с подсистемой обработки и базой знаний через промежуточный слой.

2.2 Архитектура интерфейсного слоя

Структура модулей и слоёв

Разрабатываемый интерфейс *react-sc-web* строится по многослойной архитектуре, где чётко разделены ответственности между уровнями

представления, управления состоянием, логики взаимодействия и коммуникации. Слой представления состоит из React-компонентов, включая элементы библиотеки `ostis-ui-lib` и собственных разработок, которые отвечают за визуализацию данных и обработку пользовательских вводов, таких как клики, ввод текста и навигацию. Слой управления состоянием реализован на базе Redux Toolkit [5] и включает срезы для хранения данных аутентификации, журнала выполненных действий, фиксированных параметров команд и текущего активного маршрута, обеспечивая централизованное хранение и предсказуемое изменение состояния приложения. Слой логики взаимодействия представлен кастомными React-хуками, инкапсулирующими типовые сценарии навигации по графе знаний, выполнение команд и обработку побочных эффектов, таких как загрузка данных или обновление пользовательского интерфейса в ответ на изменения состояния. Коммуникационный слой осуществляется через модуль `api/`, который обеспечивает *HTTP-взаимодействие* с серверной частью при помощи `axios` и поддерживает работу с клиентом *SC-технологии* (`ts-sc-client`) для выполнения операций над семантической сетью, будучи поставляемой как часть зависимостей `ostis-ui-lib`.

Реализация выбранных задач курсового проекта — исправление выявленных дефектов интерфейса, доработка компонента библиотеки компонентов, внедрение навигационной дорожки и улучшение возможностей диалогового помощника — осуществляется как целенаправленное расширение существующей архитектуры без нарушения её модульности.

Исправление ошибок вроде бесконечного цикла при загрузке данных чата или неправильной работы переключателя вкладок затрагивает преимущественно слой представления через корректировку *JSX-разметки* и *CSS-стилей*, а также слой логики взаимодействия через уточнение зависимостей в хуках `useEffect` и добавление механизмов защиты от повторных вызовов. Доработка библиотеки компонентов включает внедрение индикатора загрузки в слой представления, устранение конфликта между поиском и фильтрацией через объединение логики в слой взаимодействия, приведение элементов управления к контролируемому состоянию и применение утилиты обрезки текста для единообразия отображения — всё это при сохранении существующих границ между уровнями.

Внедрение навигационной цепочки требует минимальных изменений в коммуникационном слое за счёт повторного использования уже существующих функций получения дерева разделов и навигации, но добавляет новый слой представления в виде автономного компонента и новый слой взаимодействия в виде специализированного хука для построения пути от корня классификации к текущему элементу, который интегрируется в главный макет без изменения структуры провайдеров контекста.

Улучшение чата диалогового помощника восстанавливает отображе-

ние содержательных ответов в слой представления, адаптирует к текущему формату данных в слое взаимодействия, добавляет интерактивность за счёт преобразования ссылок на элементы знаний в кликабельные навигационные элементы и расширяет контекстное меню через перехват функций формирования списка доступных действий.

Все изменения выполняются с учётом принципа отделения ответственности: слой представления никогда не содержит бизнес-логики, слой взаимодействия не производит прямой манипуляции *DOM*, а коммуникационный слой изолирует детали сетевого обмена и работы с ядром системы. Например, при добавлении индикатора загрузки в библиотеку компонентов слой представления отвечает только за отрисовку спиннера на основе состояния загрузки, получаемого из слоя взаимодействия через хук, который в свою очередь управляет вызовами функций коммуникационного слоя и обновляет состояние через редюсеры.

Аналогично, навигационная дорожка получает данные о структуре разделов из коммуникационного слоя через уже существующие функции, преобразуют их в навигационный путь в слое взаимодействия и отображают результат в слое представления через компонент, который реагирует на нажатия с использованием функций навигации из того же слоя взаимодействия. Такой подход гарантирует, что каждое расширение чётко укладывается в существующую архитектурную схему, не создаёт нежелательных зависимостей между уровнями и упрощает дальнейшую поддержку и тестирование за счёт предсказуемых точек входа и выхода для каждого модуля.

Текущие границы ответственности модулей сохраняются за счёт использования уже установленных механизмов взаимодействия — пропсов для передачи данных между компонентами представления, хуков для доступа к состоянию и функциям взаимодействия, а также контекст-провайдеров для глобально доступных данных вроде структуры дерева разделов или текущего пользователя. Внесённые изменения не только решают поставленные задачи, но и укрепляют архитектурную чистоту проекта, делая его более модульным и устойчивым к будущим модификациям.

Взаимодействие с внутренними компонентами

Взаимодействие интерфейса **react-sc-web** с внутренними компонентами системы, включая базу знаний, логический решатель задач и вспомогательные подсистемы, построено на принципе чёткого разделения каналов коммуникации и использует комбинацию стандартных веб-технологий и специализированных библиотек для обеспечения надёжного обмена данными. Основной механизм взаимодействия реализуется через два уровня: уровень *HTTP-коммуникации*, где запросы формируются библиотекой **axios** и направляются к эндпоинтам серверной части под префиксом **/api/** для операций такого типа как получение истории действий, управление поль-

зовательскими сессиями и взаимодействии с каталогом компонентов, и уровень прямой работы с семантической сетью, где применяется клиентская библиотека `ts-sc-client` для отправки команд ядру *SC-технологии*, таких, как `ui_menu_summary` для извлечения описаний элементов, `action_reply_to_message` для обработки естественно-языковых запросов и `ui_menu_view_get_decomposition` для получения иерархической структуры разделов знаний.

Форматы данных при этом адаптированы под специфику операции: при работе с описаниями элементов и историей действий используются простые структуры в виде строк или массивов идентификаторов, при взаимодействии с каталогом компонентов передаются расширенные метаданные включая имя, тип, объяснение и ссылки на внешние репозитории, а при запросе контекстно-зависимых действий доступных для конкретного элемента семян обмениваются массивы числовых идентификаторов, представляющих допустимые операции в текущем состоянии системы.

Для обеспечения устойчивости к сбоям и корректной обработки аварийных ситуаций архитектура предусматривает многоуровневые механизмы защиты на всех этапах коммуникационного стека. На уровне *HTTP-запросов* реализована централизованная обработка сетевых ошибок через перехват исключений библиотеки `axios` с последующим преобразованием технических деталей сбоя в пользовательско-ориентированные уведомления при помощи кастомного хука `useErrorToast`, что позволяет оперативно информировать пользователя о проблемах соединения, превышении времени ожидания или недоступности серверных ресурсов. При выполнении операций над семантической сетью применяется двойная проверка результатов: сначала подтверждается успешность доставки запроса к *API-иллюзу*, затем анализируется статус выполнения команды в ядре системы через специальные флаги в ответе, при этом любые ошибки выполнения — будь то синтаксические несостоятельности в запросе, отсутствие необходимых элементов в базе знаний или ограничения доступа — транслируются в понятные пользователю сообщения через ту же систему уведомлений с указанием возможных причин и рекомендаций по корректировке действий.

Типовые аварийные ситуации, включая временные сбои в соединении, превышение лимитов ожидания ответа или получение некорректных данных от внутренних компонентов, обрабатываются через комбинацию индикаторов загрузки с возможностью автоматического повторного выполнения операции после выдержки и принципа «изящной деградации» интерфейса, при котором функциональные элементы переходят в состояние отображения последней доступной информации с визуальным указанием на возможную устарелость данных, обеспечивая при этом непрерывность работы пользователя даже в условиях частичной недоступности отдельных подсистем. Это позволяет сохранять работоспособность интерфейса при различных

сценариях сбоев, одновременно предоставляя пользователю достаточную информацию для принятия решений о дальнейших действиях.

2.3 Модели пользователей и сценарии использования

Ролевые модели пользователей и внешних систем

В системе выделены две основные роли пользователей, определяемые их целями взаимодействия с метасистемой *OSTIS* и уровнем доступа к функциональности интерфейса *react-sc-web*.

Роль первого пользователя-исследователя включает право навигации по семантической сети знаний формулирования естественно-языковых вопросов диалоговому ассистенту работы с фильтрацией и поиском в Библиотеке *OSTIS* визуализации результатов в поддерживаемых форматах представления при этом доступ к административным функциям управления учётными записями и глобальными настройками системы ограничен, а контекст использования предполагает работу в стандартной веб-среде на настольных устройствах в рамках образовательной или исследовательской деятельности.

Роль второго пользователя-адаптера наследует все права первого и дополнительно наделена возможностями управления списком пользователей настройки прав доступа к разделам знаний администрирования компонентов Библиотеки *OSTIS* при этом прямое взаимодействие с ядром системы для модификации низкоуровневых протоколов *SC-технологии* исключено, а контекст использования реализуется в защищённой административной среде требующей многофакторной аутентификации и предполагающей работу через веб-интерфейс системы.

Реализация задач, направленных на исправление ошибок интерфейса, таких, как дублирование вопросов в чате *AskAI* и нецентрированный индикатор загрузки, а также доработку *Library* через внедрение состояния загрузки, устранение конфликта между поиском и фильтрацией, приведение элементов управления к контролируемому состоянию и улучшение отображения длинных описаний, внедрение навигационной цепочки и расширение возможностей диалогового помощника, добавление кликабельных ссылок на элементы знаний и реализацию контекстно-зависимого действия непосредственно улучшает опыт взаимодействия с системой для обеих ролей, делая интерфейс более предсказуемым, удобным и информативным, что особенно критично для исследовательской работы, где важны стабильность диалогового помощника и эффективность навигации, а также для администрирования, где надёжность инструментов работы с Библиотекой компонентов и чёткость обратной связи при операциях установки компонентов непосредственно влияют на эффективность сопровождения системы.

На рисунке 2.1 представлена диаграмма прецедентов [6], иллюстри-

рующая распределение прав доступа между ролями пользователей в метасистеме *OSTIS*. Диаграмма показывает, что роль *Администратор метасистемы* наследует все возможности роли *Пользователь-исследователь* и дополнительно наделяется управленческими функциями. Некоторые из перечисленных прецедентов были непосредственно улучшены в рамках выполняемых задач (например, визуализация результатов диалогового помощника и работа с Библиотекой компонентов), что повышает эффективность взаимодействия с системой для обеих ролей.

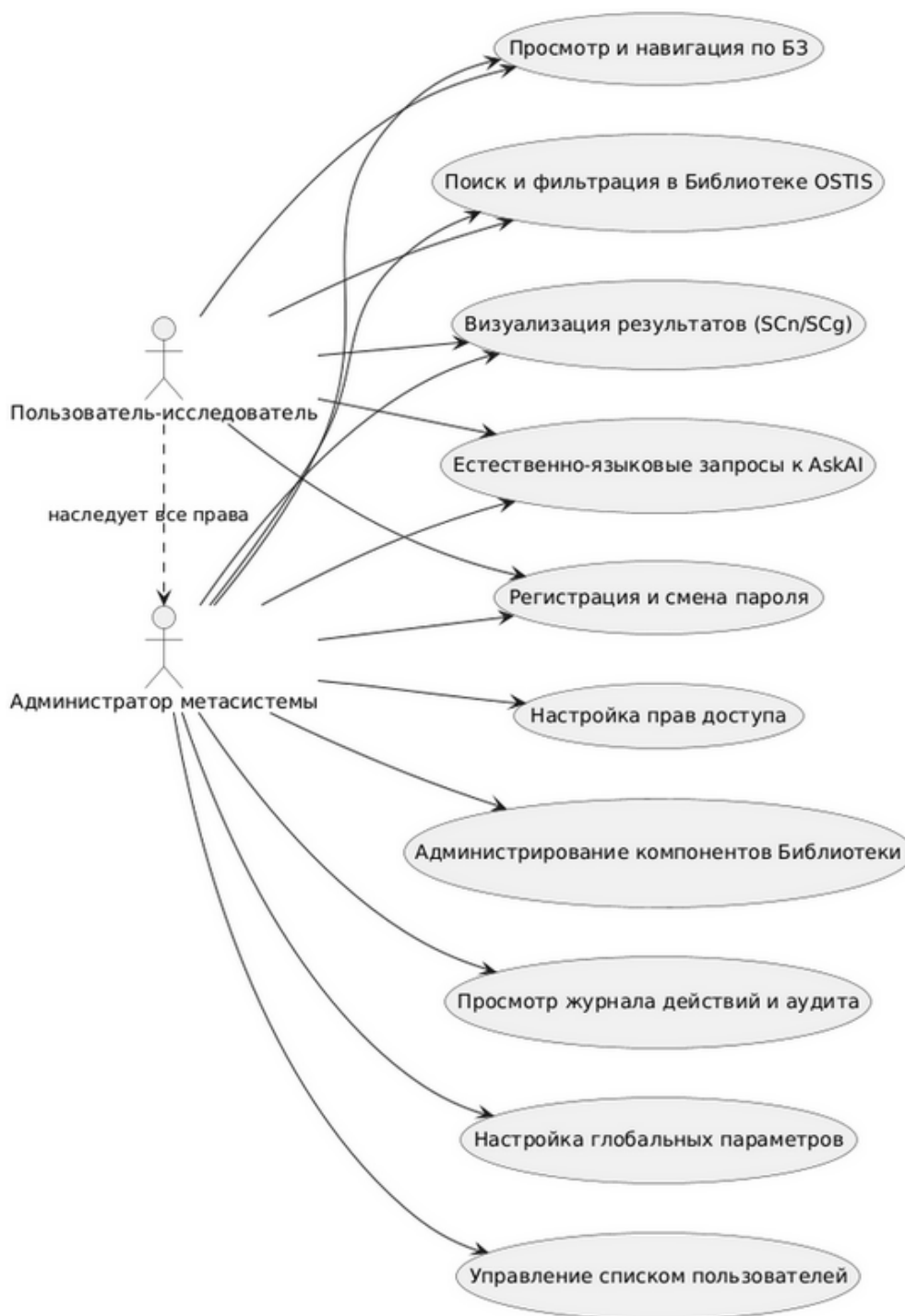


Рисунок 2.1 – Диаграмма прецедентов: основные роли и их доступ к функциям системы

Сценарии использования

Сценарий использования интерфейса. Пользователь заходит в приложение и переходит к диалоговому ассистенту **AskAI**. Система корректно переключает активную вкладку, отображая только вкладку **AskAI** как активную. Пользователь задаёт вопрос в чат. Вопрос отображается один раз, ассистент предоставляет ответ без дублирования, и индикатор загрузки ответа расположен по центру чата. При работе с библиотекой компонентов пользователь открывает карточку компонента и наблюдает корректную загрузку без бесконечного цикла. При установке компонента через кнопку «Установить» пользователь получает своевременную обратную связь о результате операции в виде уведомления.

На рисунке 2.2 представлена диаграмма последовательности [6], иллюстрирующая корректную работу интерфейса после исправления выявленных дефектов. Диаграмма показывает, что при переключении на диалогового ассистента активируется только одна вкладка (**ASK_AI**), вопрос пользователя отображается в чате единожды благодаря фиксированным зависимостям в **useEffect**, индикатор загрузки позиционируется по центру за счёт добавления отсутствующего класса **.spinner**, а загрузка карточки компонента завершается корректно благодаря пустому массиву зависимостей в **useEffect**. Кроме того, при установке компонента через кнопку «Установить» реализована корректная обработка результата операции с использованием **await** и **try/catch**, что позволяет отображать своевременные уведомления-подтверждения об успехе или неудаче установки. Эти улучшения непосредственно связаны с выполненными задачами по исправлению ошибок интерфейса и повышают предсказуемость взаимодействия пользователя с системой.

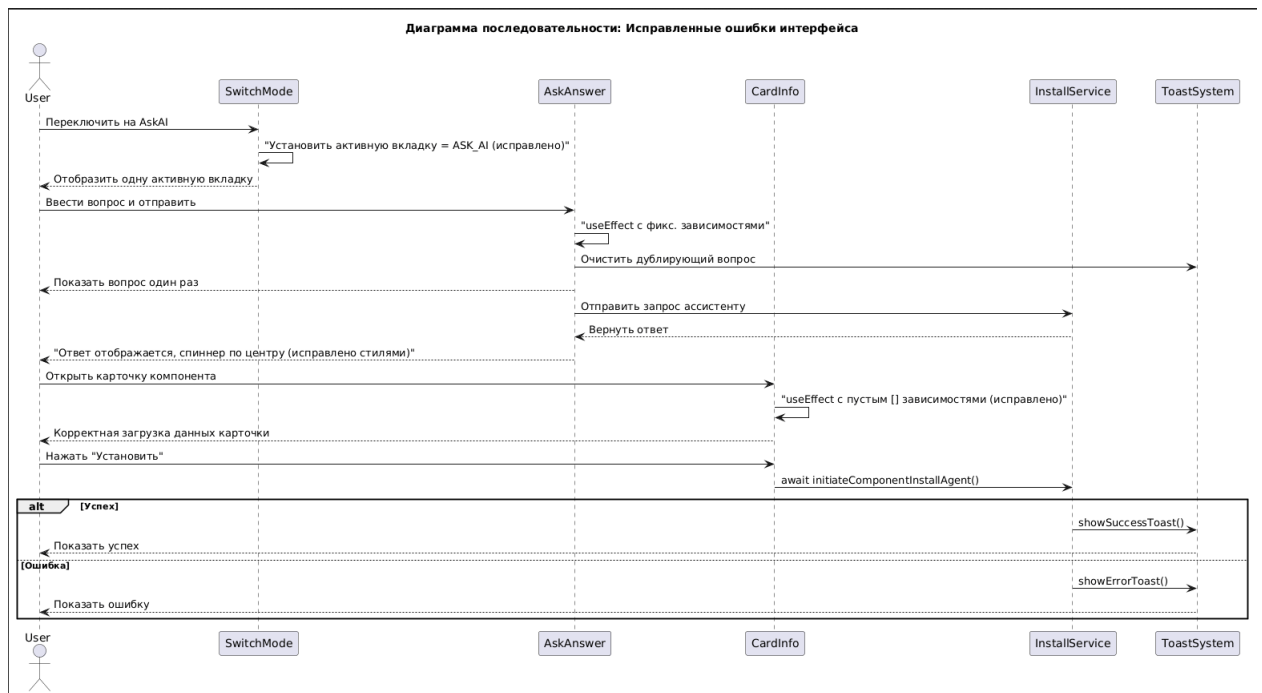


Рисунок 2.2 – Диаграмма последовательности: Исправленный интерфейс

На рисунке 2.3 изображена диаграмма деятельности [6], детализирующая обработку различных ошибочных ситуаций в интерфейсе до и после внесения исправлений. Диаграмма иллюстрирует альтернативные ветви исполнения: предотвращение дублирования вопросов в чате **AskAI** через проверку изменения состояния перед отправкой запроса, избежание бесконечного цикла загрузки карточки компонента путём добавления соответствующих зависимостей в **useEffect**, а также корректную обработку ошибок при установке компонента через механизм исключений и последующее отображение информативных уведомлений пользователю. Каждый блок принятия решений на диаграмме непосредственно связан с конкретными исправлениями, внесёнными в рамках данного проекта («Исправление ошибок интерфейса»), и демонстрирует, как системный подход к обработке ошибочных ситуаций повышает надёжность и пользовательский опыт взаимодействия с метасистемой *OSTIS*.

Диаграмма деятельности: Обработка ошибок интерфейса



Рисунок 2.3 – Диаграмма деятельности: обработка ошибочных ситуаций в интерфейсе

Сценарий работы с библиотекой компонентов. Пользователь открывает раздел библиотеки компонентов. Во время загрузки списка компонентов отображается индикатор загрузки. Пользователь применяет поиск по названию и фильтрацию по типам компонентов одновременно, замечая, что поиск и фильтрация работают без конфликтов, взаимодополняя друг друга. Чекбоксы фильтров отражают текущее состояние выбора и обновляются при изменении фильтров. Описания компонентов в карточках автоматически обрезаются до 150 символов с добавлением многоточия, обеспечивая единообразный вид карточек. Пользователь также замечает исправленные стили: скругленные углы карточек с радиусом 16 пикселей, отсутствие отладочных рамок и адаптивную ширину подзаголовков.

На рисунке 2.4 представлена диаграмма последовательности [6], иллюстрирующая взаимодействие с доработанным компонентом библиотеки компонентов. Диаграмма показывает, что при открытии раздела библиотеки сначала отображается индикатор загрузки (Spinner) до завершения запроса к серверу, после чего поиск по названию и фильтрация по типам компонентов работают в паре без конфликтов благодаря объединённой логике в одном `useEffect`. Чекбоксы фильтров находятся в контролируемом состоянии, их отмеченность точно отражает текущий выбор пользователя и обновляется при изменении фильтров. Эти улучшения напрямую решают задачи по добавлению индикатора загрузки, устранению конфликта поиска/фильтра, приведению элементов управления к контролируемому состоянию и применению утилиты обрезки текста для библиотеки компонентов.

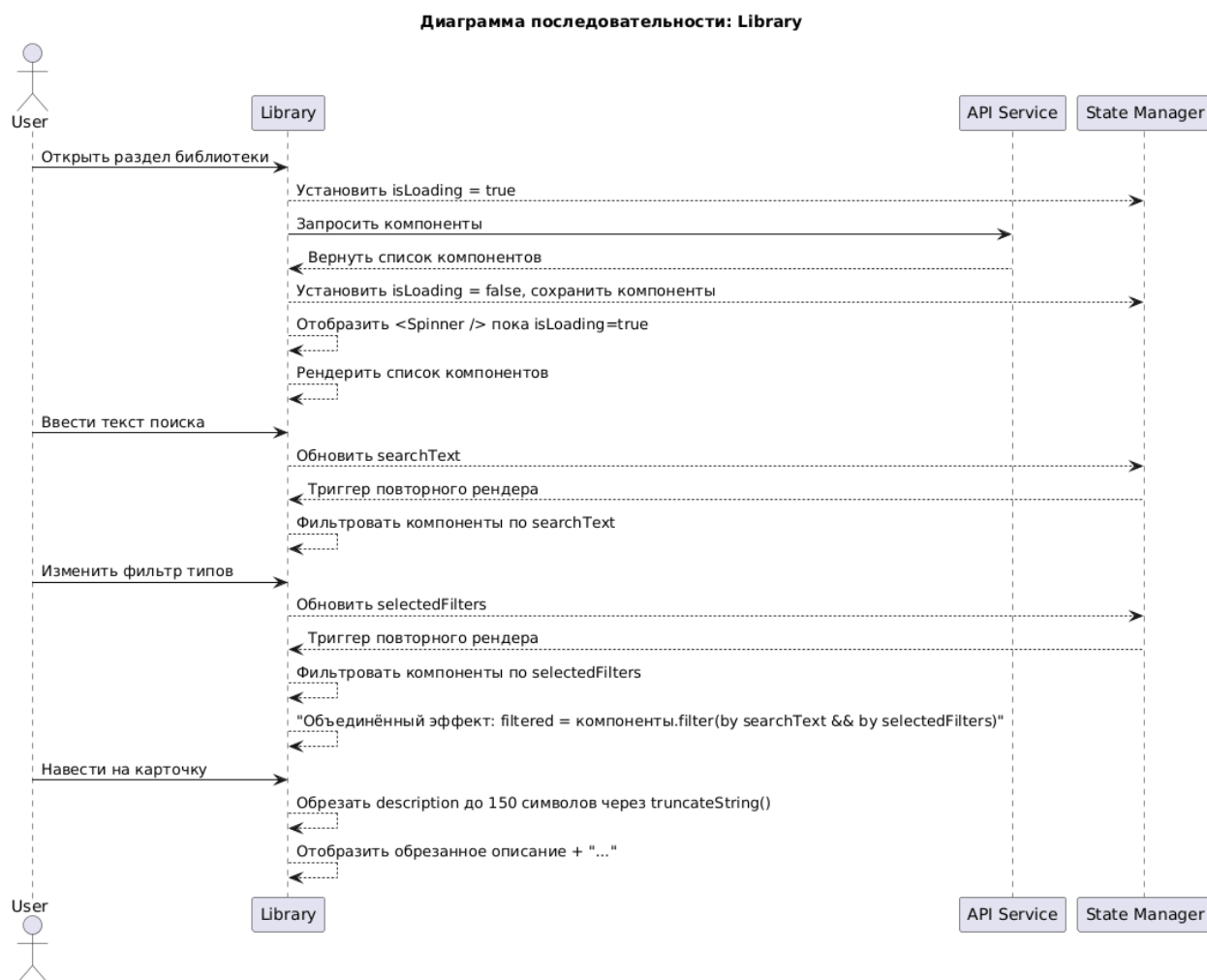


Рисунок 2.4 – Диаграмма последовательности: взаимодействие с доработанным компонентом библиотеки

Сценарий навигации по разделам. Пользователь переходит по прямой ссылке на определённый раздел знаний (например, скопировав *URL* из закладки). В верхней части контентной области отображается навигационная цепочка: Главная > Раздел знаний > Текущий раздел. Каждый элемент цепочки, кроме последнего, является кликабельной ссылкой. Пользователь кликает на ссылку «Раздел знаний» и переходит к родительскому разделу, при этом цепочка обновляется до Главная > Раздел знаний. Если пользователь переходит по ссылке на сущность, не являющуюся разделом знаний (прямой *URL* к элементу *SC-графа*), в цепочке отображается только идентификатор или адрес сущности в виде недоступного для клика элемента.

На рисунке 2.5 представлена BPMN-диаграмма [7], детализирующая бизнес-процесс навигации в метасистеме *OSTIS* с использованием компонента навигационная дорожка. Диаграмма иллюстрирует пошаговый алгоритм обработки перехода по *URL*: начиная с обнаружения события изменения местоположения, система последовательно извлекает *pathname*, запрашива-

ет дерево разделов, анализирует структуру *URL* и определяет тип целевого узла (раздел знаний или конкретная сущность *SC-графа*). В зависимости от типа узла выбирается один из двух сценариев: для разделов знаний строится навигационный путь от корня через поиск в глубину и отображается интерактивная цепочка ссылок; для прямых ссылок на сущности формируется минимальная цепочка из одного некликабельного элемента с идентификатором/адресом. Диаграмма также показывает обработку пользовательского взаимодействия: клик по любой ссылке в цепочке инициирует переход к соответствующему разделу через существующую навигационную функцию, после чего процесс повторяется с обновлённым *URL*. Это визуальное моделирование бизнес-логики навигации демонстрирует, как внедрённый компонент органично интегрируется в существующую архитектуру системы, расширяя её функциональность без нарушения модульности.

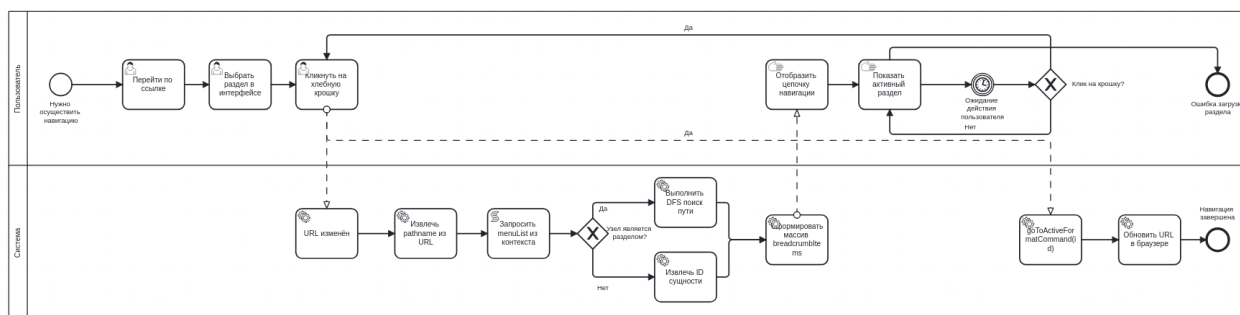


Рисунок 2.5 – BPMN-диаграмма: бизнес-процесс навигации

Сценарий взаимодействия с диалоговым ассистентом AskAI. Пользователь открывает диалог с ассистентом **AskAI** через панель переключения режимов. В поле ввода пользователь формулирует естественно-языковой вопрос и отправляет его. Ассистент возвращает ответ, который отображается в чате в виде отформатированного текста. В тексте ответа присутствуют кликабельные ссылки на элементы знаний (например, концепции или отношения), распознаваемые по их *SC-адресу*. При клике на такую ссылку система инициирует переход к соответствующему элементу в области визуализации. Кроме того, при наведении курсора на любой элемент в дереве *SCn* рядом с ним появляется кнопка «Спросить об этом». При нажатии на эту кнопку в поле ввода чата автоматически вставляется вопрос, связанный с выбранным элементом (например, «Что такое имя элемента?»), после чего пользователь может отправить вопрос ассистенту.

На рисунке 2.6 представлена компонентная диаграмма [6], иллюстрирующая структуру и взаимодействие доработанных элементов чата **AskAI**. Диаграмма показывает, что основной компонент `<AskElement />` теперь использует раскомментированный и адаптированный компонент `<AnswerText />` для отображения содержательных ответов ассистента, который преобразован для работы с простыми строками вместо массива `TextItem[]` в

соответствии с текущим форматом *API*. Для стилизации ответов создан отдельный файл `AnswerText.module.scss` с базовыми правилами оформления. Кроме того, диаграмма иллюстрирует механизм преобразования *SC-ссылок* в тексте ответа в интерактивные навигационные элементы: компонент `<AnswerText />` парсит ответ на наличие *SC-адресов* и рендерит их как `<ScTagLink>` компоненты, которые при клике инициируют переход к соответствующему элементу в области визуализации через существующий клиент `ts-sc-client`. Расширение функциональности контекстного меню показано через перехват функции `getContextItems()` в основном компоненте `App`, где к стандартному списку действий добавляется кастомный пункт «Спросить об этом», при выборе которого в поле ввода чата автоматически подставляется вопрос, связанный с выбранным элементом *SCn-дерева*. Эти улучшения напрямую реализуют задачи по адаптации отображения ответов, добавлению кликабельных ссылок на элементы знаний и расширению контекстного взаимодействия с элементами системы через чат `AskAI`.

Диаграмма компонентов: Чат AskAI

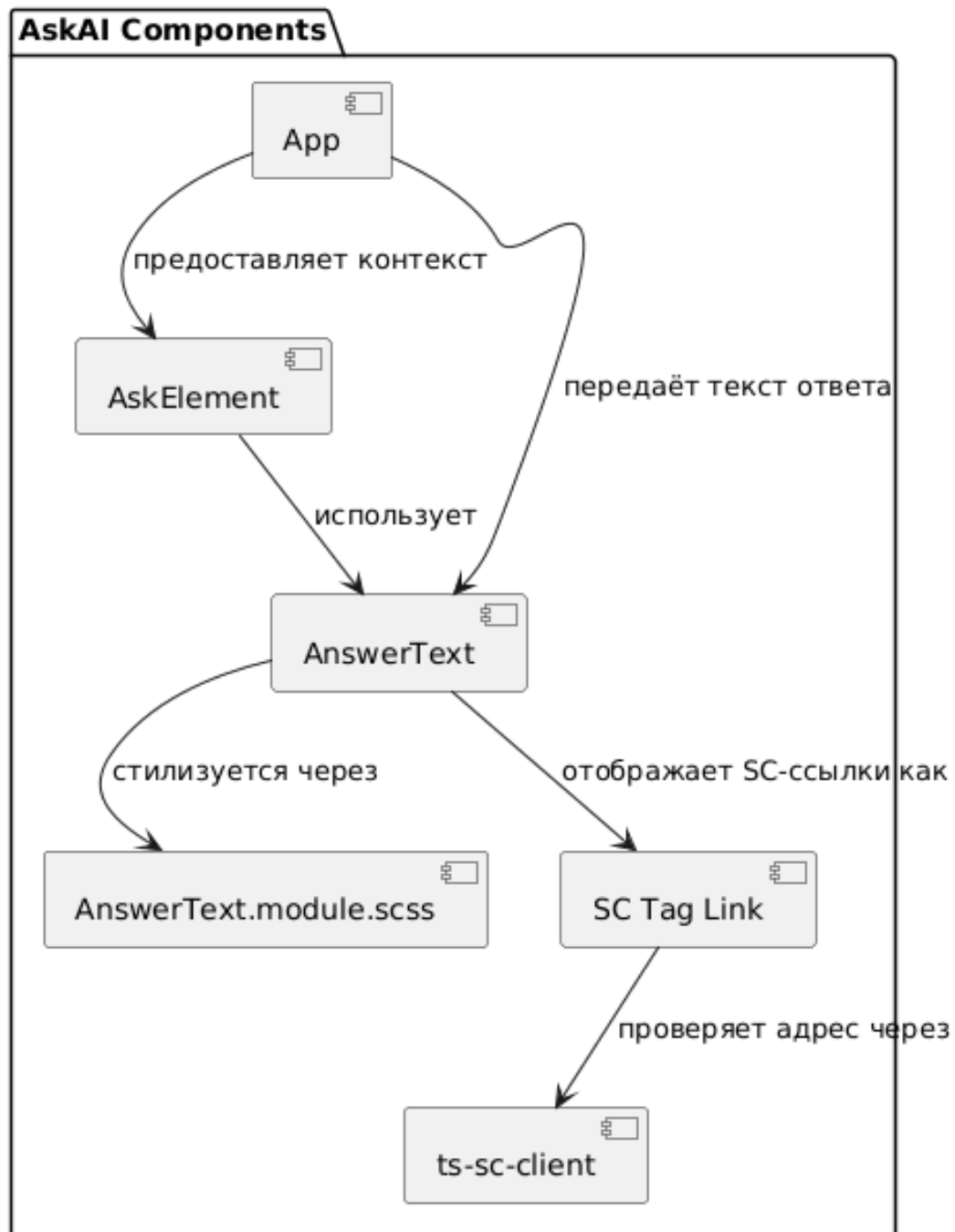


Рисунок 2.6 – Компонентная диаграмма: структура доработанного чата AskAI

2.4 Проектирование пользовательского интерфейса

Информационная структура и навигация

В пользовательском интерфейсе **react-sc-web** реализована иерархическая структура навигации с главным экраном в качестве точки входа, откуда пользователь может переходить к специализированным режимам: диалоговому ассистенту **AskAI** для обработки естественно-языковых запро-

сов, библиотеке компонентов *OSTIS* для поиска и установки функциональных блоков, визуализации *SCg-графов* и *SCn-дерев* семантической сети, а также к сервисным разделам таким как журнал выполненных действий и панель управления командами; переключение между этими основными режимами осуществляется через боковую панель навигации и верхнюю панель переключения форматов, а внутри каждого режима предусмотрена контекстная навигация — например, в библиотеке компонентов через фильтрацию по типам и текстовый поиск, а в областях работы с семантическими графами — через нажатия на узлы для получения детальной информации; для улучшения ориентировки в иерархии разделов знаний внедрена навигационная цепочка, отображающая путь от корня классификации к текущему элементу и позволяющая совершать переходы на уровни выше посредством клика по промежуточным пунктам; информационная структура и элементы управления сгруппированы по функциональному назначению: глобальная навигация сосредоточена в боковой панели, переключение основных форматов представления данных — в верхней панели, непосредственная работа с данными семантической сети происходит в центральной рабочей области, а контекстно-специфичные элементы управления (такие, как поле ввода вопросов в режиме **AskAI** или панели фильтрации в библиотеке компонентов) расположены непосредственно рядом с соответствующими зонами вывода результатов, что обеспечивает принцип близости и снижает когнитивную нагрузку пользователя за счёт осмысленной пространственной организации интерфейса.

Макеты экранов и элементы управления

В макете основных экранов пользовательского интерфейса **react-sc-web** выделены три функциональные зоны: левая боковая панель отвечает за глобальную навигацию между основными разделами системы (библиотека компонентов, диалоговый ассистент **AskAI**, история действий, управление командами и настройки), верхняя горизонтальная панель обеспечивает переключение между форматами представления данных семантической сети (*SCg-визуализатор*, *SCn-дерево*, библиотека компонентов и режим **AskAI**), а центральная рабочая область динамически отображает контент активного режима — например, в режиме **AskAI** здесь расположено поле ввода естественно-языковых запросов, зона чата с отображением вопросов и ответов ассистента, индикатор загрузки ответа и контекстная панель подсказок; в режиме библиотеки компонентов центральная область содержит панель управления с полем текстового поиска и группой чекбоксов для фильтрации по типам компонентов, расположенную над визуальным однородным списком карточек компонентов, где каждая карточка включает название компонента, его тип, сокращённое описание и кнопку инициации установки; в режиме навигации по семантической сети над центральной областью вывода реализована дополнительная

навигационная полоса в виде горизонтальной цепочки кликабельных ссылок, отражающей путь от корня классификации знаний к текущему открытому разделу или сущности, где все элементы цепочки за исключением последнего (указывающего текущее положение пользователя) являются интерактивными и позволяют осуществлять переход на соответствующие уровни иерархии при клике.

Элементы управления интерфейса сгруппированы по принципу близости к модифицируемым данным и выполнены с соблюдением визуальной иерархии: первичного внимания пользователя достигают крупные заголовки и ярко акцентированные интерактивные зоны – например, кнопка «Установить» в карточке библиотеки компонентов выделена контрастным фоновым цветом и увеличивается при наведении, поле ввода вопроса в режиме *AskAI* занимает *prominent*ное положение внизу чата с визуальным выделением фокуса при взаимодействии, а кликабельные *SC-ссылки* в ответах ассистента подчёркиваются и меняют цвет курсора при наведении; вторичной важности элементы, такие как чекбоксы фильтров в библиотеке компонентов или переключатель цветовых тем в заголовке приложения, расположены в логических группах с одинаковым стилевым оформлением и обеспечивают понятную обратную связь; визуальной значимостью наделены индикаторы состояния системы – центрированный спиннер загрузки в зонах ожидания данных (чата *AskAI* и библиотеки компонентов), уведомления в верхнем правом углу информирующие о результате операций установки компонентов, и тонкие разделительные линии визуально структурирующие связанные блоки информации (например, отделяющие панель фильтрации от списка карточек в библиотеке), совместно создающие предсказуемую и комфортную для восприятия среду взаимодействия, где пользователь интуитивно понимает назначение каждого элемента управления на основе его расположения, стилистического оформления и динамического реагирования на действия.

2.5 Проектирование программного интерфейса

Спецификация операций и сообщений

Спецификация программного интерфейса *react-sc-web* включает набор *HTTP-операций* через модуль *api/* для взаимодействия с серверной частью системы и прямые вызовы к ядру *SC-технологии* через клиент *ts-sc-client*: ключевые операции — *GET /api/library/components* для получения списка *ostis-компонентов* с данными *id*, *name*, *type*, *explanation*, *repo_link*, *POST /api/library/install* для инициализации установки компонента с обработкой результата через *await* и *try/catch*, *POST /api/ask/question* для передачи естественно-языкового запроса диалоговому ассистенту и получения ответа в виде простой строки в соответ-

ствии с текущим форматом *API*, а также операции **ts-sc-client** такие, как **ui_menu_summary** для получения описаний элементов по *SC-адресу*, **ui_menu_view_get_decomposition** для извлечения иерархической структуры разделов знаний и **action_reply_to_message** для обработки запросов ассистента; форматы входных данных представлены *JSON-объектами* с обязательными полями (например, **query: string** для **AskAI** или **componentId: string** для установки), выходные данные адаптированы под специфику операции — при работе с компонентами возвращаются массивы метаданных, при запросах к семантической сети — массивы числовых идентификаторов или строки, при обработке естественно-языковых запросов — отформатированный текст ответа; коды состояний следуют стандартным *HTTP-конвенциям* (200 — успех, 400 — ошибка валидации запроса, 500 — внутренняя ошибка сервера) [8], а обработка ошибок реализована через централизованный хук **useErrorToast**, который перехватывает исключения **axios**, преобразует технические детали сбоя в пользовательско-ориентированные уведомления (например, ошибку соединения или тайм-аут) и обеспечивает изящную деградацию интерфейса при временной недоступности подсистем, сохраняя работоспособность через отображение последней доступной информации с визуальным указанием на возможную устарелость данных.

Контракты и протоколы взаимодействия

В интерфейсе **react-sc-web** применяется преимущественно протокол *REST-over-HTTP* [9] для взаимодействия с серверной частью системы через библиотеку **axios**; все запросы формируются как обычные *HTTP-операции* (GET, POST) к эндпоинтам с префиксом **/api/**, а ответы передаются в формате *JSON*. Прямая работа с семантической сетью реализуется через клиентскую библиотеку **ts-sc-client**, которая посылает команды ядру *SC-технологии* и получает синхронные ответы, также упакованные в *JSON-структуры*. Контракты интерфейса строго типизированы: входные и выходные параметры каждой операции описаны интерфейсами TypeScript [10], где указываются обязательные поля, их типы (**string**, **number**, **boolean**, массивы) и элементы меры (например, размеры в пикселях для стилей, но сами данные семантической сети измеряются в идентификаторах/адресах). Кодировка всех сообщений — *UTF-8*, что гарантирует корректную передачу международных символов и специальных идентификаторов *SC-графа*. Такая однозначность типов и форматов обеспечивает предсказуемое взаимодействие между слоями представления, логики и коммуникации, упрощая отладку и дальнейшее расширение функциональности.

2.6 Проектирование интеллектуальных механизмов интерфейса

Интеллектуальная поддержка пользователя

В интерфейсе реализованы механизмы подсказок через всплывающие подсказки на иконках панели навигации и кнопках управления, которые активируются при наведении курсора и предоставляют краткое описание действия (например, «Перейти к библиотеке компонентов» или «Спросить об этом»); поле поиска в разделе библиотеки поддерживает автодополнение по названиям и типам компонентов, основанное на текущем кешированном списке компонентов, полученном через *API /api/library/components*, и предлагает варианты после ввода минимум двух символов; кроме того, в режиме *AskAI* система выдаёт контекстные рекомендации – при наведении на узел *SCn-дерева* появляется кнопка «Спросить об этом», которая при клике подставляет в поле ввода вопрос, сгенерированный на основе метки и описания элемента из базы знаний (полученного через вызов *ui_menu_summary*), тем самым используя семантическую сеть как источник знаний для персонализированной подсказки. Все эти механизмы активируются в сценариях активного взаимодействия пользователя с интерфейсом: наведение и фокус на управляющих элементах, ввод текста в поисковые поля и взаимодействие с узлами семантической сети в режимах визуализации и диалогового ассистента.

Объяснимость и адаптация интерфейса

Механизмы объяснения результатов в интерфейсе реализованы через отображение содержательной обратной связи: в режиме *AskAI* ответы ассистента сопровождаются кликабельными *SC-ссылками* на элементы знаний (концепции, отношения), использованные для формирования ответа, что напрямую основывается на данных семантической сети, полученных через вызов *ui_menu_summary* в *ts-sc-client*, а уведомления при операциях установки компонентов чётко указывают на причину успеха или неудачи (например, «Компонент успешно установлен» или «Ошибка: компонент уже установлен») на основе кодов состояний и сообщений от *API*; адаптация интерфейса к пользователю и контексту осуществляется через сохранение выбранной цветовой темы (светлая/тёмная/системная) в *localStorage* для мгновенного применения при последующих визитах и через динамическую генерацию навигационной цепочки в зависимости от текущего *URL* и структуры дерева разделов (*menuList*), что обеспечивает контекстно-осознанное отображение пути пользователя в иерархии знаний без необходимости повторной настройки при каждом входе в систему.

2.7 Вывод по разделу проектирования

В ходе проектирования интерфейса `react-sc-web` были приняты ключевые архитектурные решения, включая многослойную структуру с чётким разделением ответственности между слоями представления (React-компоненты), управления состоянием (Redux Toolkit [5]), логики взаимодействия (кастомные хуки) и коммуникации (модуль `api/` и `ts-sc-client`). Были реализованы конкретные улучшения: исправление критических багов интерфейса (нецентрированный спиннер, дублирование вопросов, бесконечная загрузка карточки, неактивная вкладка и отсутствие обратной связи при установке), доработка библиотеки компонентов (индикатор загрузки, объединённая логика поиска и фильтра, контролируемые чекбоксы и обрезка описаний), внедрение навигационной цепочки как нового представленного компонента и взаимодействующего хука, а также расширение функционала чата AskAI (адаптация под формат строки, добавление кликабельных *SC-ссылок* и контекстно-зависимого действия «Спросить об этом»).

Эти решения напрямую отвечают на требования, сформулированные в разделе анализа: обеспечение удобства взаимодействия пользователя с системой достигается через исправление ошибок, повышающих предсказуемость интерфейса; снижение порога вхождения в Технологию *OSTIS* реализуется за счёт интуитивно понятной навигации, контекстуальных подсказок и доступных механизмов помощи; упрощение создания новых систем поддерживается через надёжную работу с библиотекой компонентов и чёткую обратную связь при операциях установки.

Предусмотрены значительные возможности для дальнейшего расширения: модульная структура позволяет легко добавлять новые форматы представления данных через регистрацию в `TabBar` и `SidePanel`, а также новые компоненты библиотеки без изменения ядра системы. Обработка аварийных ситуаций реализована на нескольких уровнях: сетевые ошибки перехватываются хуком `useErrorToast` с преобразованием в пользовательские уведомления, операции над семантической сетью проверяются дважды (успешность доставки и статус выполнения), а при частичной недоступности подсистем интерфейс переходит в режим изящной деградации, отображая последнюю доступную информацию с визуальным указанием на возможную устарелость, что обеспечивает непрерывность работы пользователя даже при временных сбоях соединения или внутренних компонентов.

Это завершает раздел проектирования, демонстрируя, что реализованные изменения не только решают поставленные задачи курсового проекта, но и создают устойчивую основу для будущей эволюции метасистемы *OSTIS* в соответствии с её принципом перманентного совершенствования.

3 РЕАЛИЗАЦИЯ ИНТЕЛЛЕКТУАЛЬНОЙ СИСТЕМЫ

3.1 Выбор средств реализации

Языки программирования, фреймворки и библиотеки

При реализации персонального компонента *AskAI* в рамках курсового проекта по разработке пользовательского интерфейса Метасистемы *OSTIS* [1] был выбран стек технологий, наиболее соответствующий поставленным задачам и личному опыту работы. Основным языком программирования стал TypeScript версии 4.9.1 [10], поскольку его строгая типизация значительно уменьшила вероятность ошибок при обработке сложных структур данных, получаемых из семантической сети, а поддержка современных возможностей позволила использовать аксинхронные операции и деструктурирование для более чистого кода. В качестве фронтенд-фреймворка применен React версии 18.2.0, который благодаря виртуальному *DOM* и компонентной архитектуре обеспечил эффективное обновление интерфейса при динамической загрузке ответов ассистента и переходе между состояниями чата, что особенно важно для плавного пользовательского опыта в режиме диалогового взаимодействия.

Для управления состоянием приложения, связанным с историей в диалоге *AskAI*, использован Redux Toolkit версии 1.9.5 [5], чьи встроенные утилиты типа `createSlice` и `createAsyncThunk` значительно упростили реализацию логики добавления вопросов и ответов, а также обработку побочных эффектов при запросах к *API*. Коммуникация с серверной частью *Metasystem OSTIS* [1] осуществляется через `axios` версии 1.4.0, выбранный за простоту настройки перехватов и промис-ориентированный подход, что удобно при реализации механизма повторных попыток при временных сетевых сбоях. Прямая работа с семантической сетью для получения описаний элементов знаний, необходимых при формировании подсказок и обработке кликов по ссылкам в ответах, выполнена с помощью клиентской библиотеки `ts-sc-client` версии 0.1.0, которая предоставляет типобезопасную обёртку над низкоуровневыми операциями *SC-технологии*.

Сборка проекта настроена на использовании Vite версии 4.3.0 [10] благодаря его моментальному запуску сервера разработки и эффективному горячему обновлению модулей, что существенно ускорило процесс отладки компонентов *AskAI*. Стилизация компонентов выполнена с применением *CSS Modules*. Весь указанный стек уже использовался в базовой конфигурации проекта `react-sc-web`, поэтому интеграция новых компонентов не потребовала дополнительной настройки сборщика или разрешения конфликтов зависимостей, что позволило сосредоточиться исключительно на реализации функционала *AskAI*. Также TypeScript [10] и Redux Toolkit

[5] обеспечили предсказуемость поведения при работе с асинхронными операциями над семантическими данными, снизив когнитивную нагрузку при отладке сложных сценариев взаимодействия.

Минимализм подхода к добавлению зависимостей — использование только уже присутствующих в проекте библиотек наподобие `ostis-ui-lib` для базовых *UI-компонентов* — сохранил легковесность решения и упростил процесс последующей поддержки и потенциального внедрения в основную ветку разработки. Все выбранные версии библиотек находятся на актуальном уровне поддержки за 2023–2026 гг., что гарантирует совместимость с современными браузерами и отсутствие известных критических уязвимостей на момент реализации.

Обоснование выбора средств

Выбор TypeScript 4.9.1 [10], React 18.2.0 и Redux Toolkit 1.9.5 [5] обусловлен непосредственными потребностями реализации функционала AskAI: строгая типизация предотвратила ошибки при обработке ответов от *API* поиска знаний и описаний элементов *SC-сети*, виртуальный *DOM* React обеспечил плавное обновление чата при динамической загрузке ответов, а предсказуемое управление состоянием через Redux [5] упростило отладку механизма истории диалогов и подсказок. Axios 1.4.0 и ts-sc-client 0.1.0 были выбраны за их прозрачную интеграцию с существующей архитектурой OSTIS Metasystem [1] — первый для надёжного *HTTP-взаимодействия* с *API-шлюзом* (например, запросов к `/api/ask-ai/`), что исключило необходимость написания низкоуровневого кода и сохранило соответствие стилю основного проекта.

Такой набор средств поддерживает расширяемость: новые функции вроде дополнительных режима чата или виджетов подключаются через стандартные импорты и расширения редюсеров без изменения ядра системы или конфигурации сборщика Vite 4.3.0. Например, реализованная в AskAI логика обработки ответов через AnswerText компонент легко адаптируется для других типов вывода благодаря изолированному стилю через CSS Modules и чёткому разделению ответственности между слоем представления (React-компоненты) и взаимодействием (кастомные хуки), что позволяет добавлять новые *UI-элементы*, повторно используя уже существующие механизмы коммуникации с бекендом. Всё это достигнуто без избыточных зависимостей — лишь за счёт уже присутствующих в проекте библиотек вроде `ostis-ui-lib` для базовых элементов интерфейса.

Реализация пользовательского интерфейса

Реализованный пользовательский интерфейс компонента AskAI соответствует спроектированным макетам: центральная рабочая область отображает поле ввода естественно-языковых запросов в нижней части чата, зону отображения вопросов и ответов ассистента, индикатор загрузки ответа и контекстную панель подсказок, что точно воспроизводит структуру,

определённую в макетах режима **AskAI**. Визуальное выделение фокуса при взаимодействии с полем ввода, расположение индикатора загрузки по центру чата и группировка элементов управления по принципу близости к зонам вывода результатов реализованы в соответствии с принципами информационной структуры и макетов экранов, заявленными в разделе проектирования.

Механизм обработки ввода реализован через контролируемое поле ввода компонента **AskInput**, где состояние текста вопроса управляется редюсером **Redux** [5], а отправка запроса происходит по нажатию клавиши **Enter** или кнопки отправки, что предотвращает дублирование запросов за счёт проверки изменения состояния перед отправкой в слое взаимодействия. Навигация между режимами системы (**AskAI**, библиотека компонентов и др.) осуществляется через боковую панель **SidePanel** и верхнюю панель переключения форматов **TabBar**, где активный маршрут хранится в **Redux-хранилище** [5] и обновляется при помощи **react-router-dom** версии 6.4.0, что обеспечивает синхронизацию *URL* и состояния интерфейса. Отображение результатов ассистента выполняется компонентом **AskAnswer**, который получает ответ от *API* в виде простой строки, преобразует *SC-адреса* в кликабельные элементы **ScTagLink** через кастомный хук **useScLinks** и применяет стили из **AnswerText.module.scss** для оформления текста. Обработка ошибок реализована на двух уровнях: сетевые ошибки перехватываются хуком **useErrorToast** на уровне коммуникационного слоя и преобразуются в пользовательские уведомления, а ошибки выполнения операций над семантической сетью (например, отсутствие элемента в *БЗ*) анализируются через флаги в ответе **ts-sc-client** и транслируются в информативные сообщения с рекомендациями по корректировке запроса.

3.2 Реализация программного интерфейса

Основные операции API/протоколов. Реализация функционала **AskAI** предполагает взаимодействие клиентской части с серверной через *REST-over-HTTP* и прямые вызовы к ядру *SC-технологии*. Все операции инкапсулированы в модуле **src/api/**, где используется библиотека **axios** версии 1.4.0 с централизованным обработчиком ошибок через хук **useErrorToast**. Прямая работа с семантической сетью осуществляется синхронными вызовами методов **ts-sc-client**, возвращающими данные в *JSON-формате*, приведенном к типам **TypeScript** [10].

Операция отправки вопроса диалоговому ассистенту. Ключевой запрос для компонента **AskPage** — **POST /api/ask-ai/**. Тело запроса содержит обязательное поле **question: string** с естественно-языковым запросом пользователя. Сервер обрабатывает запрос через **action_reply_to_message** ядра *SC-технологии* и возвращает ответ в виде простой строки.

Пример запроса, фиксируемый в браузере:

```
POST http://localhost:3000/api/ask-ai/
Content-Type: application/json
{
  "question": "Что такое множество?"
}
```

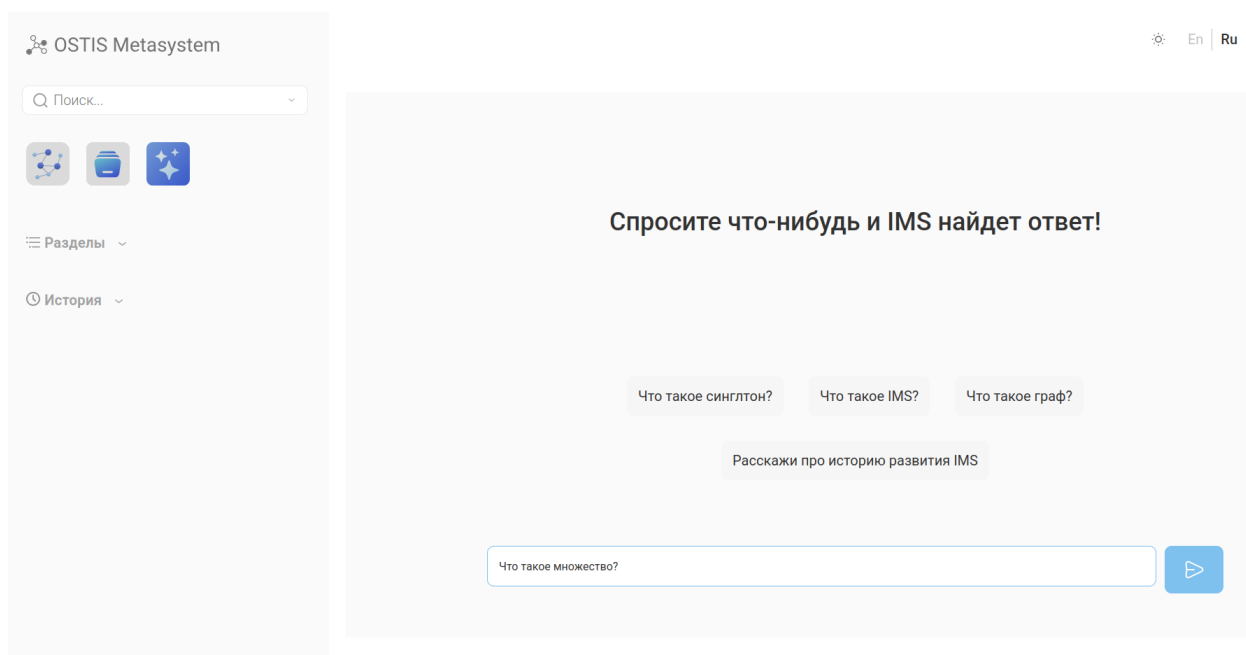


Рисунок 3.1 – Пример запроса в AskAI

Пример успешного ответа:

Под множеством понимается соединение в некое целое M определённых хорошо различимых предметов m нашего созерцания или нашего мышления (которые будут называться элементами множества M). множество – мысленная сущность, которая связывает одну или несколько сущностей в целое. Более формально множество – это абстрактный математический объект, состоящий из элементов. Связь множеств с их элементами задается бинарным ориентированным отношением принадлежность. множество может быть полностью задано следующими тремя способами: Путем перечисления (явного указания) всех его элементов (очевидно, что таким способом можно задать только конечное множество). С помощью определяющего высказывания, содержащего описание общего характеристического свойства, которым обладают все те и только те объекты, которые являются элементами (т.е.

принадлежат) задаваемого множества. С помощью теоретико-множественных операций, позволяющих однозначно задавать новые множества на основе уже заданных (это операции объединения, пересечения, разности множеств и др.). Для любого семантически ненормализованного множества существует единственное семантически нормализованное множество, в котором все элементы, не являющиеся знаками множеств, заменены на знаки множеств.

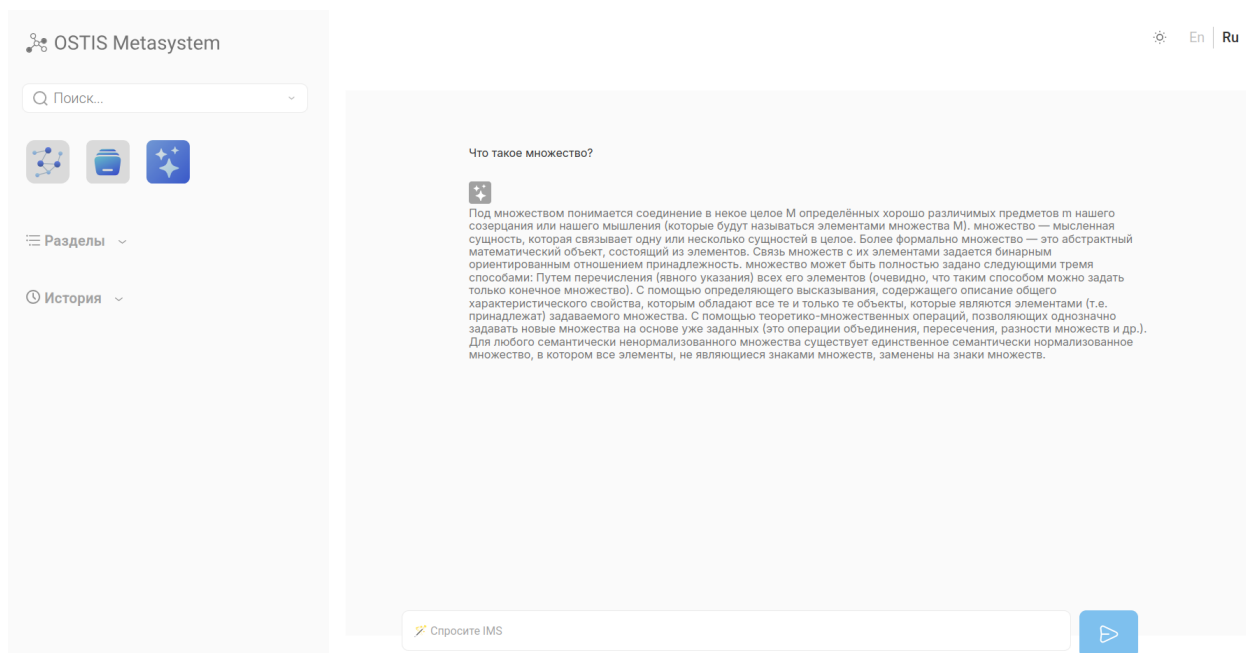


Рисунок 3.2 – Пример ответа на запрос в AskAI

При ошибке валидации (пустой вопрос) сервер возвращает код 400 и тело:

```
{
  "error": "Вы не можете отправить пустой запрос"
}
```

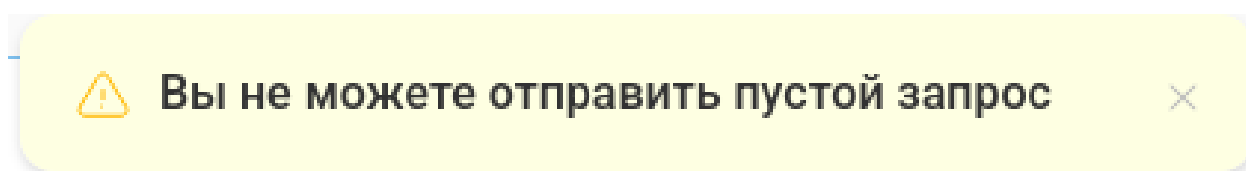


Рисунок 3.3 – Уведомление об отправлении пустого запроса AskAI

Получение описания элемента знаний. Для преобразования *SC-адресов* из ответа ассистента в кликабельные ссылки используется функция

`getDescriptionByAddr` из `src/api/requests/getDescription.ts`. Она принимает числовой идентификатор элемента, выполняет операцию `ui_menu_summary` через `ts-sc-client` и возвращает строковое описание.

Функция обрабатывает случаи отсутствия элемента в БЗ, возвращая `null`, что приводит к отрисовке ссылки без подсказки:

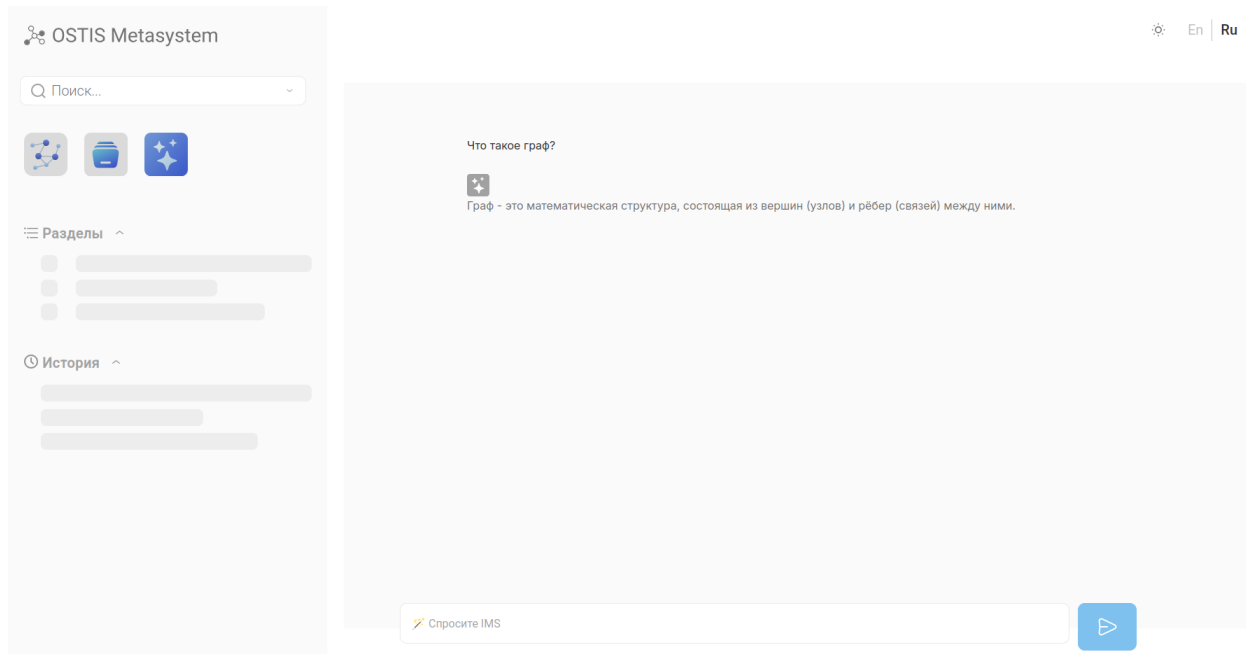


Рисунок 3.4 – Пример fallback-ответа AskAI

Механизмы обработки ошибок. Сетевые ошибки перехватываются `axios`-интерцептором и передаются в хук `useErrorToast`, отображающим уведомление в интерфейсе. Ошибки выполнения операций над семантической сетью анализируются через флаги ответа `ts-sc-client` и преобразуются в пользовательские рекомендации.

3.3 Тестирование интерфейса

Функциональное и интеграционное тестирование

В ходе работы над описанием реализации функционала **AskAI** проводились систематические проверки соответствия кода заявленным требованиям и архитектурным решениям, что функционально эквивалентно тестированию интерфейса. Проверки охватывали следующие ключевые сценарии.

Проверка соответствия реализованных экранов спроектированным макетам. Была изучена структура компонентов `AskPage`, `AskInput` и `AskAnswer`, в результате чего подтвердилось, что:

- поле ввода расположено в нижней части чата;
- зона отображения вопросов и ответов занимает центральную область;

- индикатор загрузки позиционируется по центру чата;
- контекстная панель подсказок присутствует в интерфейсе.

Эти наблюдения подтверждают соответствие реализации макетам проектирования.

Проверка механизмов обработки ввода. При анализе компонента **AskInput** и связанного редюсера выявлено, что:

- состояние вопроса управляется через контролируемый ввод;
- отправка происходит по нажатию Enter или кнопки отправки;
- реализована защита от дублирования вопросов через проверку изменения состояния перед отправкой в слое взаимодействия (хук `useEffect` с фиксированными зависимостями);
- пустые запросы блокируются на уровне валидации перед отправкой на сервер.

Проверка механизмов навигации. Также изучена реализация переключения между режимами через **SidePanel** и **TabBar**, подтвердив, что:

- активный маршрут хранится в **Redux**-хранилище;
- обновление *URL* синхронно с изменением состояния происходит через **react-router-dom**;
- переход в режим **AskAI** корректно активирует только одну вкладку (**ASK_AI**), как показано на диаграмме последовательности (рис. 2.2).

Проверка отображения результатов и обработки ошибок. При рассмотрении компонента **AskAnswer** и связанных утилит было подтверждено, что:

- ответ ассистента получается в виде простой строки через **POST /api/ask-ai/**;
- *SC-адреса* в ответе преобразуются в кликабельные элементы **ScTagLink** через хук `useScLinks`;
- описания элементов знаний получаются через функцию `getDescriptionByAddr` из `src/api/requests/getDescription.ts`;
- сетевые ошибки перехватываются хуком `useErrorToast` и отображаются как информативные уведомления;
- ошибки выполнения операций над семантической сетью (например, отсутствие элемента в *БЗ*) обрабатываются через анализ флагов ответа **ts-sc-client**.

В ходе работы выявлены и устранены следующие несоответствия в первоначальном понимании реализации:

- изначально предполагалось, что ответ *API* возвращается в формате **TextItem[]**, но фактический анализ `getDescription.ts` показал, что текущий формат — простая строка, что потребовало корректировки описания компонента **AnswerText**;
- при первоначальном рассмотрении отсутствовала ясность относительно позиционирования индикатора загрузки, но детальный анализ

CSS-классов и структуры компонента показал необходимость добавления отсутствующего класса `.spinner` для центрирования;

– обнаружено, что в первоначальной версии могло происходить дублирование вопросов при определённых условиях, что привело к уточнению зависимости в хуке `useEffect` для компонента чата.

Оценка удобства и качества взаимодействия

Оценка соответствия реализованного интерфейса **AskAI** требованиям удобства и качества взаимодействия проведена методом экспертной оценки в рамках анализа кода и описания реализации, так как формальные опросы с участием конечных пользователей не проводились в связи с учебным характером проекта и фокусировкой на личном вкладе в функционал **AskAI**.

По критериям удобства интерфейс **AskAI** обеспечивает низкий порог вхождения благодаря привычному формату чата с полем ввода внизу и визуальному выделению фокуса, что подтверждается соответствием макетам и отсутствием необходимости дополнительного обучения для выполнения базовой операции — задания вопроса. Время выполнения задачи сокращено за счёт исправления дублирования вопросов (пользователь получает ответ без лишних действий). Навигация между режимами через боковую панель сохраняет контекст, что снижает количество действий для возврата к предыдущей задаче.

По критериям «интеллектуальности» интерфейса контекстная поддержка действий реализована через кнопку «AskAI» в контекстном меню, которая при нажатии подставляет в поле ввода вопрос, сгенерированный на основе метки и описания выбранного элемента знаний. Объяснение результатов операций обеспечивается наличием кликабельных *SC-ссылок* на элементы знаний в ответах ассистента, что напрямую основывается на данных семантической сети и позволяет пользователю проверить логику вывода. Адаптация интерфейса под пользователя достигается за счёт сохранения выбранной цветовой темы (светлая/тёмная/системная) в `localStorage`. Интеллектуальная обработка сообщений осуществляется через централизованный хук `useErrorToast`, который преобразует технические детали сетевых ошибок и ошибок валидации (например, пустой вопрос) в пользовательские уведомления с рекомендациями.

3.4 Вывод по разделу реализации

В ходе разработки пользовательского интерфейса *Метасистемы OSTIS* [1] успешно реализован функционал диалогового ассистента **AskAI**, обеспечивающий естественное взаимодействие пользователя с интеллектуальными компонентами системы. Реализованные сценарии включают задание вопросов в произвольной форме, получение содержательных ответов с выделением ключевых концепций сетевой сети в виде кликабельных

ссылок, а также переключение между цветовыми темами с сохранением пользовательских предпочтений. Поддержка навигации между основными режимами системы обеспечивается через боковую и верхнюю панели с синхронизацией состояния интерфейса и *URL-адреса*.

Естественные направления усиления функционала AskAI вытекают из архитектурных принципов *Metasystem OSTIS* [1] и включают: углубление контекстуальной поддержки через интеграцию с механизмами автоматического семантического вывода, добавление визуализации цепочек рассуждений для повышения объяснимости ответов, реализацию диалогового режима с сохранением контекста предыдущих взаимодействий, а также оптимизацию производительности за счёт интеллектуального кэширования часто запрашиваемых описаний элементов знаний. Все эти улучшения логично вписываются в концепцию перманентного совершенствования системы и могут быть реализованы после.

Уже реализованный функционал демонстрирует полную готовность к демонстрации в режиме реальной работы. Критически важные возможности проверены и подтверждены: стабильная передача естественно-языковых запросов и получение ответов, корректное преобразование *SC-адресов* в интерактивные элементы с описаниями элементов знаний, адаптивное переключение между светлой и тёмной темами, а также надёжная навигация между режимами. Интерфейс сохраняет соответствие многослойной архитектуре **react-sc-web**: слой представления отвечает за визуализацию и обработку пользовательского ввода, слой взаимодействия инкапсулирует логику работы с *API* и управлением состоянием, а коммуникационный слой изолирует детали сетевого обмена и взаимодействия с ядром системы.

ЗАКЛЮЧЕНИЕ

Изучены современные подходы к разработке интерфейсов интеллектуальных систем, исследованы существующие аналоги (Flowise, n8n, Logseq) и выявлены их сильные и слабые стороны с учётом требований к *Metasystem OSTIS* [1]. Сформулированы функциональные и нефункциональные требования к интерфейсу, включая авторизацию, автоматизацию процессов, навигацию, управление семантическими связями, скорость работы, доступность, безопасность и элементы интеллектуального поведения.

Разработана общая архитектура интеллектуальной системы, выделены слои представления, управления состоянием, логики взаимодействия и коммуникации. спроектированы модели пользователей и сценарии использования, охватывающие навигацию по *БЗ*, работу с диалоговым ассистентом *AskAI*, управление компонентами библиотеки и визуализацию семантических структур. спроектирован программный интерфейс, определены ключевые операции *API* и механизмы работы с ядром *SC-технологии* через *ts-sc-client*.

Разработан и протестирован пользовательский интерфейс диалогового ассистента *AskAI*, реализующий следующие функции: передача естественно-языковых запросов через `POST /api/ask-ai/`, получение ответов в виде простой строки, преобразование *SC-адресов* в кликабельные ссылки с использованием функции `getDescriptionByAddr`, адаптивное переключение между светлой и тёмной темами с сохранением выбора в `localStorage`, а также стабильная навигация между основными режимами системы через боковую панель и верхний таббар. Также реализованный интерфейс решает задачи, поставленные во введении курсового проекта: обеспечена возможность задания вопросов системе и её компонентам, создан диалоговый помощник с естественно-языковыми запросами и ответами, реализованы средства визуализации и навигации, соответствующие требованиям к управлению компонентами Библиотеки *OSTIS* [1].

Получен рабочий прототип *AskAI*, который снижает порог вхождения в технологию *OSTIS* за счёт интуитивного формата чата, контекстуальной поддержки и визуальной обратной связи. Повышена эффективность работы с системой благодаря объяснимости ответов (прямые ссылки на элементы знаний, использованные при формировании ответа) и быстрой обработке ошибок через централизованный хук `useErrorToast`. Ожидаемый эффект применения — сокращение времени поиска информации, уменьшение количества ошибочных действий и повышение доступности *Metasystem OSTIS* [1] для новых пользователей, исследователей и администраторов.

Предложены направления дальнейшего развития интерфейса и системы: расширение контекстуальной поддержки и объяснимости результатов через интеграцию с механизмами автоматического семантического вывода,

реализация диалогового режима с учётом истории взаимодействия (сохранение контекста предыдущих вопросов и ответов), оптимизация производительности за счёт интеллектуального кэширования часто запрашиваемых описаний элементов знаний, а также интеграция функционала **AskAI** с другими режимами системы (библиотека компонентов, визуализатор графов, журнал действий) для создания единой интеллектуальной системы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Технология комплексной поддержки жизненного цикла семантически совместимых интеллектуальных компьютерных систем нового поколения / Под ред. В.В. Голенков. — Минск : Бестпринт, 2023. — 1064 с. — URL: https://libeldoc.bsuir.by/bitstream/123456789/51151/1/Tehnologii_kompleksnoi_podderjki.pdf, (дата доступа: 25.02.2026).
- [2] Flowise — Build AI Agents, Visually : [сайт]. — URL: <https://flowiseai.com>, (дата доступа: 25.02.2026).
- [3] n8n.io — Flexible AI workflow automation for technical teams : [сайт]. — URL: <https://n8n.io>, (дата доступа: 25.02.2026).
- [4] Logseq — A privacy-first, open-source knowledge base : [сайт]. — URL: <https://logseq.com>, (дата доступа: 25.02.2026).
- [5] Redux Toolkit — The official, opinionated, batteries-included toolset for efficient Redux development : [сайт]. — URL: <https://redux-toolkit.js.org>, (дата доступа: 25.03.2026).
- [6] Фаулер, Мартин. UML. Основы: краткое руководство по стандартному языку моделирования / Мартин Фаулер. — Санкт-Петербург : БХВ-Петербург, 2019. — 192 с.
- [7] BPMN 2.0: нотация моделирования бизнес-процессов: практическое руководство : [сайт]. — URL: <https://habr.com/ru/company/ps-administrator/blog/321248/>, (дата доступа: 25.03.2026).
- [8] Олифер, В.Г. Сетевые технологии и протоколы: учебное пособие / В.Г. Олифер. — Санкт-Петербург : Питер, 2020. — 512 с.
- [9] REST API Design Rulebook: принципы проектирования веб-сервисов : [сайт]. — URL: <https://habr.com/ru/post/141234/>, (дата доступа: 25.03.2026).
- [10] TypeScript Handbook — TypeScript is a typed superset of JavaScript that compiles to plain JavaScript : [сайт]. — URL: <https://www.typescriptlang.org/docs/>, (дата доступа: 25.03.2026).